Unit 3: Exception Handling and File Handling

Practical 1: ATM Transaction System

Description:

Develop an ATM simulation that handles invalid PINs, insufficient balance, invalid withdrawal amounts, and transaction failures using exception handling.

Concepts Covered

- try
- except
- finally
- User-defined Exceptions

Input:

```
print("Name              :- JETHAVA JAY VINODBHAI")
print("Enrollment No. :- 250283152004\n")


class InvalidPIN(Exception):
    pass


class InsufficientBalance(Exception):
    pass


class InvalidWithdrawal(Exception):
    pass


class TransactionFailed(Exception):
    pass


class ATM:
    def __init__(self, pin, balance):
        self.pin = pin
        self.balance = balance

    def withdraw(self, amount):
        if amount <= 0:
            raise InvalidWithdrawal("Withdrawal amount must be greater than 0.")

        if amount > self.balance:
            raise InsufficientBalance("Insufficient balance.")

        if amount % 100 != 0:
            raise InvalidWithdrawal("Amount must be a multiple of 100.")

        self.balance -= amount
        print("Transaction successful!")
        print("Amount withdrawn: ₹", amount)
        print("Remaining balance: ₹", self.balance)
```

```
atm = ATM(1234, 10000)

print("===== ATM TRANSACTION SYSTEM =====")

try:
    entered_pin = int(input("Enter your PIN: "))

    if entered_pin != atm.pin:
        raise InvalidPIN("Invalid PIN!")

    print("\nPIN verified successfully.")
    print("Current Balance: ₹", atm.balance)

    amount = int(input("Enter withdrawal amount: ₹"))

    atm.withdraw(amount)

except InvalidPIN as e:
    print("Error:", e)

except InsufficientBalance as e:
    print("Error:", e)

except InvalidWithdrawal as e:
    print("Error:", e)

except TransactionFailed as e:
    print("Error:", e)

except ValueError:
    print("Error: Please enter a valid numeric value.")

except Exception as e:
    print("Unexpected Error:", e)

finally:
    print("\nThank you for using the ATM.")
    print("Transaction process completed.")
```

Output:

```
Name             :- JETHAVA JAY VINODBHAI
Enrollment No. :- 250283152004

===== ATM TRANSACTION SYSTEM =====
Enter your PIN: 1234

PIN verified successfully.
Current Balance: ₹ 10000
Enter withdrawal amount: ₹3000
Transaction successful!
Amount withdrawn: ₹ 3000
Remaining balance: ₹ 7000

Thank you for using the ATM.
Transaction process completed.

================================ RESTART: D:/P
Name             :- JETHAVA JAY VINODBHAI
Enrollment No. :- 250283152004

===== ATM TRANSACTION SYSTEM =====
Enter your PIN: 1111
Error: Invalid PIN!

Thank you for using the ATM.
Transaction process completed.
```

Practical 2: Student Report Card File System

Description:

Store student records in a text file and provide options to add, search, modify, and display records.

Concepts Covered

- Text Files
- File Handling
- with Statement

Input:

```
print("Name            :- JETHAVA JAY VINODBHAI")
print("Enrollment No. :- 250283152004\n")

FILE_NAME = "students.txt"

def add_student():
    roll_no = input("Enter Roll Number: ")
    name = input("Enter Student Name: ")
    marks = input("Enter Marks: ")

    with open(FILE_NAME, "a") as file:
        file.write(f"{roll_no},{name},{marks}\n")

    print("Student record added successfully.")

def display_students():
    try:
        with open(FILE_NAME, "r") as file:
            records = file.readlines()

            if not records:
                print("No student records found.")
                return

            print("\n===== STUDENT RECORDS =====")
            for record in records:
                roll_no, name, marks = record.strip().split(",")
                print("Roll No :", roll_no)
                print("Name    :", name)
                print("Marks   :", marks)
                print("-------------------------")

    except FileNotFoundError:
        print("No student records found.")
```

```python
def search_student():
    roll_no = input("Enter Roll Number to search: ")
    found = False

    try:
        with open(FILE_NAME, "r") as file:
            for record in file:
                data = record.strip().split(",")

                if data[0] == roll_no:
                    print("\nStudent Found!")
                    print("Roll No :", data[0])
                    print("Name    :", data[1])
                    print("Marks   :", data[2])
                    found = True
                    break

        if not found:
            print("Student record not found.")

    except FileNotFoundError:
        print("No student records found.")
```

```python
def modify_student():
    roll_no = input("Enter Roll Number to modify: ")
    found = False

    try:
        with open(FILE_NAME, "r") as file:
            records = file.readlines()

        with open(FILE_NAME, "w") as file:
            for record in records:
                data = record.strip().split(",")

                if data[0] == roll_no:
                    name = input("Enter New Student Name: ")
                    marks = input("Enter New Marks: ")

                    file.write(f"{roll_no},{name},{marks}\n")
                    found = True
                else:
                    file.write(record)

        if found:
            print("Student record modified successfully.")
        else:
            print("Student record not found.")

    except FileNotFoundError:
        print("No student records found.")
```

```
while True:
    print("\n===== STUDENT REPORT CARD SYSTEM =====")
    print("1. Add Student")
    print("2. Search Student")
    print("3. Modify Student")
    print("4. Display All Students")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()

    elif choice == "2":
        search_student()

    elif choice == "3":
        modify_student()

    elif choice == "4":
        display_students()

    elif choice == "5":
        print("Thank you for using Student Report Card System.")
        break

    else:
        print("Invalid choice! Please try again.")
```

Output:

```
===== STUDENT REPORT CARD SYSTEM =====
1. Add Student
2. Search Student
3. Modify Student
4. Display All Students
5. Exit
Enter your choice: 1
Enter Roll Number: 07
Enter Student Name: jay
Enter Marks: 80
Student record added successfully.

===== STUDENT REPORT CARD SYSTEM =====
1. Add Student
2. Search Student
3. Modify Student
4. Display All Students
5. Exit
Enter your choice: 4

===== STUDENT RECORDS =====
Roll No : 07
Name    : jay
Marks   : 80
-------------------------
```

Practical 3: Hospital Patient Record System

Description:

Maintain patient records using binary files and Python Pickle for permanent storage.

Concepts Covered

- Pickle
- Binary Files

Input:

```
print("Name            :- JETHAVA JAY VINODBHAI")
print("Enrollment No. :- 250283152004\n")

import pickle

FILE_NAME = "patients.dat"

def add_patient():
    patient_id = input("Enter Patient ID: ")
    name = input("Enter Patient Name: ")
    age = int(input("Enter Age: "))
    disease = input("Enter Disease: ")
    contact = input("Enter Contact Number: ")

    patient = {
        "id": patient_id,
        "name": name,
        "age": age,
        "disease": disease,
        "contact": contact
    }

    with open(FILE_NAME, "ab") as file:
        pickle.dump(patient, file)

    print("Patient record added successfully.")

def display_patients():
    try:
        with open(FILE_NAME, "rb") as file:
            print("\n===== PATIENT RECORDS =====")

            found = False

            while True:
                try:
                    patient = pickle.load(file)

                    print("Patient ID :", patient["id"])
                    print("Name       :", patient["name"])
                    print("Age        :", patient["age"])
                    print("Disease    :", patient["disease"])
                    print("Contact    :", patient["contact"])
                    print("----------------------------")
```

```
                    found = True

                except EOFError:
                    break

            if not found:
                print("No patient records found.")

    except FileNotFoundError:
        print("No patient records found.")

def search_patient():
    patient_id = input("Enter Patient ID to search: ")
    found = False

    try:
        with open(FILE_NAME, "rb") as file:

            while True:
                try:
                    patient = pickle.load(file)

                    if patient["id"] == patient_id:
                        print("\n===== PATIENT FOUND =====")
                        print("Patient ID :", patient["id"])
                        print("Name       :", patient["name"])
                        print("Age        :", patient["age"])
                        print("Disease    :", patient["disease"])
                        print("Contact    :", patient["contact"])

                        found = True
                        break

                except EOFError:
                    break

        if not found:
            print("Patient record not found.")

    except FileNotFoundError:
        print("No patient records found.")
```

```
while True:

    print("\n===== HOSPITAL PATIENT RECORD SYSTEM =====")
    print("1. Add Patient")
    print("2. Search Patient")
    print("3. Display All Patients")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        try:
            add_patient()
        except ValueError:
            print("Error: Age must be a number.")

    elif choice == "2":
        search_patient()

    elif choice == "3":
        display_patients()

    elif choice == "4":
        print("Thank you for using Hospital Patient Record System.")
        break

    else:
        print("Invalid choice! Please try again.")
```

Output

```
Name            :- JETHAVA JAY VINODBHAI
Enrollment No. :- 250283152004


===== HOSPITAL PATIENT RECORD SYSTEM =====
1. Add Patient
2. Search Patient
3. Display All Patients
4. Exit
Enter your choice: 1
Enter Patient ID: 890
Enter Patient Name: Mx
Enter Age: 50
Enter Disease: flu
Enter Contact Number: 2828291038
Patient record added successfully.
```

Practical 4: Log File Analyzer

Description:

Read a system log file, count different types of errors, and generate a summary report while logging exceptions.

Concepts Covered

- File Handling
- Logging
- Exception Handling

Input:

```
print("Name            :- JETHAVA JAY VINODBHAI")
print("Enrollment No. :- 250283152004\n")

import logging

LOG_FILE = "system.log"
REPORT_FILE = "summary_report.txt"

logging.basicConfig(
    filename="analyzer.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

def analyze_log_file():
    error_count = 0
    warning_count = 0
    critical_count = 0
    info_count = 0

    try:
        logging.info("Starting log file analysis.")

        with open(LOG_FILE, "r") as file:
            for line in file:
                line = line.strip()

                if "ERROR" in line:
                    error_count += 1
                elif "WARNING" in line:
                    warning_count += 1
                elif "CRITICAL" in line:
                    critical_count += 1
                elif "INFO" in line:
                    info_count += 1
```

```
        with open(REPORT_FILE, "w") as report:
            report.write("===== LOG FILE SUMMARY REPORT =====\n")
            report.write(f"ERROR Count    : {error_count}\n")
            report.write(f"WARNING Count  : {warning_count}\n")
            report.write(f"CRITICAL Count : {critical_count}\n")
            report.write(f"INFO Count     : {info_count}\n")

        logging.info("Log file analysis completed successfully.")

        print("\n===== LOG FILE SUMMARY REPORT =====")
        print("ERROR Count    :", error_count)
        print("WARNING Count  :", warning_count)
        print("CRITICAL Count :", critical_count)
        print("INFO Count     :", info_count)
        print("\nSummary report generated successfully.")

    except FileNotFoundError:
        logging.error("System log file not found.")
        print("Error: system.log file not found.")

    except PermissionError:
        logging.error("Permission denied while accessing the file.")
        print("Error: Permission denied.")

    except Exception as e:
        logging.exception("Unexpected error occurred.")
        print("Unexpected Error:", e)

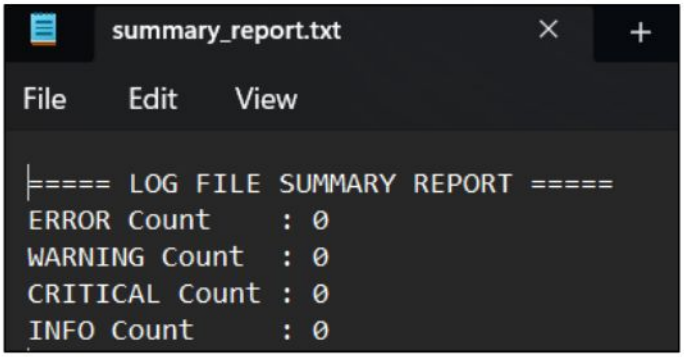
analyze_log_file()
```

Output:

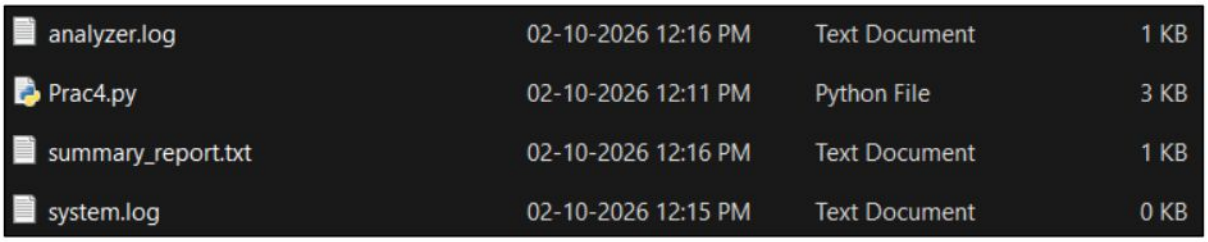
```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004


===== LOG FILE SUMMARY REPORT =====
ERROR Count    : 0
WARNING Count  : 0
CRITICAL Count : 0
INFO Count     : 0

Summary report generated successfully.
```



```
2026-10-07 20:04:44,764 - INFO - Starting log file analysis.
2026-10-07 20:04:44,764 - ERROR - System log file not found.
2026-10-07 20:05:56,563 - INFO - Starting log file analysis.
2026-10-07 20:05:56,564 - INFO - Log file analysis completed successfully.
```



analyzer.log	02-10-2026 12:16 PM	Text Document	1 KB
Prac4.py	02-10-2026 12:11 PM	Python File	3 KB
summary_report.txt	02-10-2026 12:16 PM	Text Document	1 KB
system.log	02-10-2026 12:15 PM	Text Document	0 KB

Practical 5: Employee Salary Management using Binary Files

Description:

Store employee records in binary format and provide searching, updating, and deletion facilities.

Concepts Covered

- Binary Files
- seek()
- tell()

Input:

```
print("Name            :- JETHAVA JAY VINODBHAI")
print("Enrollment No. :- 250283152004\n")

import pickle

FILE_NAME = "employees.dat"

def add_employee():
    emp_id = input("Enter Employee ID: ")
    name = input("Enter Employee Name: ")
    salary = float(input("Enter Salary: "))

    employee = {
        "id": emp_id,
        "name": name,
        "salary": salary
    }

    with open(FILE_NAME, "ab") as file:
        position = file.tell()
        pickle.dump(employee, file)

    print("Employee record added successfully.")
    print("Record stored at byte position:", position)

def search_employee():
    emp_id = input("Enter Employee ID to search: ")

    try:
        with open(FILE_NAME, "rb") as file:

            while True:
                try:
                    position = file.tell()
                    employee = pickle.load(file)

                    if employee["id"] == emp_id:
                        print("\n===== EMPLOYEE FOUND =====")
                        print("Employee ID :", employee["id"])
                        print("Name        :", employee["name"])
                        print("Salary      :", employee["salary"])
                        print("Record position:", position)
                        return
```

```
                except EOFError:
                    break

        print("Employee record not found.")

    except FileNotFoundError:
        print("Employee file not found.")

def update_salary():
    emp_id = input("Enter Employee ID to update: ")
    found = False

    try:
        with open(FILE_NAME, "rb") as file:
            employees = []

            while True:
                try:
                    employee = pickle.load(file)

                    if employee["id"] == emp_id:
                        new_salary = float(input("Enter New Salary: "))
                        employee["salary"] = new_salary
                        found = True

                    employees.append(employee)

                except EOFError:
                    break

        with open(FILE_NAME, "wb") as file:
            for employee in employees:
                pickle.dump(employee, file)

        if found:
            print("Employee salary updated successfully.")
        else:
            print("Employee record not found.")

    except FileNotFoundError:
        print("Employee file not found.")
```

```
def delete_employee():
    emp_id = input("Enter Employee ID to delete: ")
    found = False

    try:
        with open(FILE_NAME, "rb") as file:
            employees = []

            while True:
                try:
                    employee = pickle.load(file)

                    if employee["id"] == emp_id:
                        found = True
                    else:
                        employees.append(employee)

                except EOFError:
                    break

        with open(FILE_NAME, "wb") as file:
            for employee in employees:
                pickle.dump(employee, file)

        if found:
            print("Employee record deleted successfully.")
        else:
            print("Employee record not found.")

    except FileNotFoundError:
        print("Employee file not found.")
```

```
def display_employees():
    try:
        with open(FILE_NAME, "rb") as file:

            print("\n===== EMPLOYEE RECORDS =====")

            found = False

            while True:
                try:
                    position = file.tell()
                    employee = pickle.load(file)

                    print("Employee ID :", employee["id"])
                    print("Name        :", employee["name"])
                    print("Salary      :", employee["salary"])
                    print("File Position:", position)
                    print("----------------------------")

                    found = True

                except EOFError:
                    break

            if not found:
                print("No employee records found.")

    except FileNotFoundError:
        print("Employee file not found.")

while True:

    print("\n===== EMPLOYEE SALARY MANAGEMENT =====")
    print("1. Add Employee")
    print("2. Search Employee")
    print("3. Update Salary")
    print("4. Delete Employee")
    print("5. Display All Employees")
    print("6. Exit")

    choice = input("Enter your choice: ")
```

```
    try:

        if choice == "1":
            add_employee()

        elif choice == "2":
            search_employee()

        elif choice == "3":
            update_salary()

        elif choice == "4":
            delete_employee()

        elif choice == "5":
            display_employees()

        elif choice == "6":
            print("Thank you for using Employee Salary Management System.")
            break

        else:
            print("Invalid choice! Please try again.")

    except ValueError:
        print("Error: Please enter a valid numeric value.")
```

Output:

```
===== EMPLOYEE SALARY MANAGEMENT =====
1. Add Employee
2. Search Employee
3. Update Salary
4. Delete Employee
5. Display All Employees
6. Exit
Enter your choice: 1
Enter Employee ID: 005
Enter Employee Name: jay
Enter Salary: 50000
Employee record added successfully.
Record stored at byte position: 0

===== EMPLOYEE SALARY MANAGEMENT =====
1. Add Employee
2. Search Employee
3. Update Salary
4. Delete Employee
5. Display All Employees
6. Exit
Enter your choice: 5

===== EMPLOYEE RECORDS =====
Employee ID : 005
Name        : jay
Salary      : 50000.0
File Position: 0
----------------------------
```

Practical 6: Online Examination Result Backup System

Description:

Create a backup utility that stores examination results into binary files and restores them whenever required.

Concepts Covered

- Pickle
- File Handling

Input:

```
print("Name             :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import pickle

BACKUP_FILE = "exam_results.dat"

def create_backup():
    results = []
    n = int(input("Enter number of students: "))

    for i in range(n):
        print(f"\nEnter details for Student {i + 1}")

        roll_no = input("Enter Roll Number: ")
        name = input("Enter Student Name: ")
        marks = float(input("Enter Marks: "))

        result = {
            "roll_no": roll_no,
            "name": name,
            "marks": marks
        }

        results.append(result)

    with open(BACKUP_FILE, "wb") as file:
        pickle.dump(results, file)

    print("\nExamination results backed up successfully.")
```

```
def restore_backup():
    try:
        with open(BACKUP_FILE, "rb") as file:
            results = pickle.load(file)

        print("\n===== RESTORED EXAMINATION RESULTS =====")

        for result in results:
            print("Roll Number :", result["roll_no"])
            print("Name        :", result["name"])
            print("Marks       :", result["marks"])
            print("----------------------------------------")

    except FileNotFoundError:
        print("Backup file not found.")

    except EOFError:
        print("Backup file is empty or corrupted.")

    except Exception as e:
        print("Error while restoring backup:", e)
```

```
while True:

    print("\n===== ONLINE EXAMINATION RESULT BACKUP =====")
    print("1. Create Backup")
    print("2. Restore Backup")
    print("3. Exit")

    choice = input("Enter your choice: ")

    try:

        if choice == "1":
            create_backup()

        elif choice == "2":
            restore_backup()

        elif choice == "3":
            print("Thank you for using the system.")
            break

        else:
            print("Invalid choice! Please try again.")

    except ValueError:
        print("Error: Please enter a valid numeric value.")
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004


===== ONLINE EXAMINATION RESULT BACKUP =====
1. Create Backup
2. Restore Backup
3. Exit
Enter your choice: 1
Enter number of students: 2

Enter details for Student 1
Enter Roll Number: 07
Enter Student Name: jay
Enter Marks: 80

Enter details for Student 2
Enter Roll Number: 08
Enter Student Name: om
Enter Marks: 80

Examination results backed up successfully.
```

Unit 4: Classes and Object-Oriented Programming

Practical 1: Banking Management System

Description:
Develop a banking application that allows account creation, deposits, withdrawals, balance inquiry, and transaction history using classes and objects.

Concepts Covered
- Classes
- Objects
- Constructors

Input:
```
print("Name             :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

class BankAccount:

    def __init__(self, account_no, name, balance=0):
        self.account_no = account_no
        self.name = name
        self.balance = balance
        self.transactions = []

        self.transactions.append(
            f"Account created with initial balance: ₹{balance}"
        )

    def deposit(self, amount):
        if amount <= 0:
            print("Invalid deposit amount.")
            return

        self.balance += amount
        self.transactions.append(f"Deposited: ₹{amount}")

        print("Amount deposited successfully.")
        print("Current Balance: ₹", self.balance)

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdrawal amount.")
            return

        if amount > self.balance:
            print("Insufficient balance.")
            return

        self.balance -= amount
        self.transactions.append(f"Withdrawn: ₹{amount}")

        print("Amount withdrawn successfully.")
        print("Current Balance: ₹", self.balance)
```

```
    def check_balance(self):
        print("\n===== ACCOUNT BALANCE =====")
        print("Account Number :", self.account_no)
        print("Account Holder  :", self.name)
        print("Balance         : ₹", self.balance)

    def transaction_history(self):
        print("\n===== TRANSACTION HISTORY =====")

        if not self.transactions:
            print("No transactions found.")
        else:
            for i, transaction in enumerate(self.transactions, 1):
                print(i, ".", transaction)


print("===== BANKING MANAGEMENT SYSTEM =====")

account_no = input("Enter Account Number: ")
name = input("Enter Account Holder Name: ")
initial_balance = float(input("Enter Initial Balance: ₹"))

account = BankAccount(account_no, name, initial_balance)

print("\nAccount created successfully!")
```

```
while True:

    print("\n===== BANK MENU =====")
    print("1. Deposit Money")
    print("2. Withdraw Money")
    print("3. Check Balance")
    print("4. Transaction History")
    print("5. Exit")

    choice = input("Enter your choice: ")

    try:

        if choice == "1":
            amount = float(input("Enter amount to deposit: ₹"))
            account.deposit(amount)

        elif choice == "2":
            amount = float(input("Enter amount to withdraw: ₹"))
            account.withdraw(amount)

        elif choice == "3":
            account.check_balance()

        elif choice == "4":
            account.transaction_history()

        elif choice == "5":
            print("\nThank you for using Banking Management System.")
            break

        else:
            print("Invalid choice! Please try again.")

    except ValueError:
        print("Please enter a valid numeric amount.")
```

Output:

```
Name            :- JETHAVA JAY V.
Enrollment No. :- 250283152004

===== BANKING MANAGEMENT SYSTEM =====
Enter Account Number: 1
Enter Account Holder Name: JAY
Enter Initial Balance: ₹5000

Account created successfully!

===== BANK MENU =====
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Transaction History
5. Exit
Enter your choice: 1
Enter amount to deposit: ₹500
Amount deposited successfully.
Current Balance: ₹ 5500.0

===== BANK MENU =====
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Transaction History
5. Exit
Enter your choice: 2
Enter amount to withdraw: ₹600
Amount withdrawn successfully.
Current Balance: ₹ 4900.0

===== BANK MENU =====
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Transaction History
5. Exit
Enter your choice: 3

===== ACCOUNT BALANCE =====
Account Number : 1
Account Holder  : JAY
Balance         : ₹ 4900.0

===== BANK MENU =====
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Transaction History
5. Exit
```

Practical 2: Employee Management System

Description:

Create Employee and Manager classes demonstrating inheritance. Calculate salary, bonus, and department details.

Concepts Covered

- Inheritance
- Polymorphism

Input:

```
print("Name             :- JETHAVA JAY ")
print("Enrollment No. :- 250283152004\n")

class Employee:

    def __init__(self, emp_id, name, salary):
        self.emp_id = emp_id
        self.name = name
        self.salary = salary

    def calculate_salary(self):
        return self.salary

    def display_details(self):
        print("Employee ID :", self.emp_id)
        print("Name        :", self.name)
        print("Salary      : ₹", self.calculate_salary())

class Manager(Employee):

    def __init__(self, emp_id, name, salary, department, bonus):
        super().__init__(emp_id, name, salary)
        self.department = department
        self.bonus = bonus

    def calculate_salary(self):
        return self.salary + self.bonus

    def display_details(self):
        print("Manager ID     :", self.emp_id)
        print("Name           :", self.name)
        print("Department     :", self.department)
        print("Basic Salary: ₹", self.salary)
        print("Bonus          : ₹", self.bonus)
        print("Total Salary: ₹", self.calculate_salary())

print("===== EMPLOYEE MANAGEMENT SYSTEM =====")

print("\nEnter Employee Details")
emp_id = input("Enter Employee ID: ")
name = input("Enter Employee Name: ")
salary = float(input("Enter Employee Salary: ₹"))
```

```
employee = Employee(emp_id, name, salary)

print("\n===== EMPLOYEE DETAILS =====")
employee.display_details()

print("\nEnter Manager Details")
manager_id = input("Enter Manager ID: ")
manager_name = input("Enter Manager Name: ")
manager_salary = float(input("Enter Manager Basic Salary: ₹"))
department = input("Enter Department: ")
bonus = float(input("Enter Manager Bonus: ₹"))

manager = Manager(
    manager_id,
    manager_name,
    manager_salary,
    department,
    bonus
)

print("\n===== MANAGER DETAILS =====")
manager.display_details()
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== EMPLOYEE MANAGEMENT SYSTEM =====

Enter Employee Details
Enter Employee ID: 0005
Enter Employee Name: JAY
Enter Employee Salary: ₹50000

===== EMPLOYEE DETAILS =====
Employee ID : 0005
Name        : JAY
Salary      : ₹ 50000.0

Enter Manager Details
Enter Manager ID: 04
Enter Manager Name: mX
Enter Manager Basic Salary: ₹30000
Enter Department: it
Enter Manager Bonus: ₹500

===== MANAGER DETAILS =====
Manager ID     : 04
Name           : mX
Department     : it
Basic Salary: ₹ 30000.0
Bonus          : ₹ 500.0
Total Salary: ₹ 30500.0
```

Practical 3: Hospital Management System

Description:

Develop classes for Patient, Doctor, and Appointment to manage hospital records.

Concepts Covered

- OOP
- Encapsulation

Input:

```
print("Name             :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

class Patient:

    def __init__(self, patient_id, name, age, disease):
        self.__patient_id = patient_id
        self.__name = name
        self.__age = age
        self.__disease = disease

    def get_patient_id(self):
        return self.__patient_id

    def get_name(self):
        return self.__name

    def get_age(self):
        return self.__age

    def get_disease(self):
        return self.__disease

    def display_details(self):
        print("Patient ID :", self.__patient_id)
        print("Name       :", self.__name)
        print("Age        :", self.__age)
        print("Disease    :", self.__disease)
```

```
class Doctor:

    def __init__(self, doctor_id, name, specialization):
        self.__doctor_id = doctor_id
        self.__name = name
        self.__specialization = specialization

    def get_doctor_id(self):
        return self.__doctor_id

    def get_name(self):
        return self.__name

    def get_specialization(self):
        return self.__specialization

    def display_details(self):
        print("Doctor ID       :", self.__doctor_id)
        print("Name            :", self.__name)
        print("Specialization  :", self.__specialization)
```

```python
class Appointment:

    def __init__(self, appointment_id, patient, doctor, date):
        self.__appointment_id = appointment_id
        self.__patient = patient
        self.__doctor = doctor
        self.__date = date

    def display_details(self):
        print("Appointment ID :", self.__appointment_id)
        print("Date           :", self.__date)
        print("Patient        :", self.__patient.get_name())
        print("Doctor         :", self.__doctor.get_name())
        print("Specialization :", self.__doctor.get_specialization())
```

```python
print("===== HOSPITAL MANAGEMENT SYSTEM =====")
print("\nEnter Patient Details")
patient_id = input("Enter Patient ID: ")
patient_name = input("Enter Patient Name: ")
patient_age = int(input("Enter Patient Age: "))
disease = input("Enter Disease: ")

patient = Patient(
    patient_id,
    patient_name,
    patient_age,
    disease
)

print("\nEnter Doctor Details")
doctor_id = input("Enter Doctor ID: ")
doctor_name = input("Enter Doctor Name: ")
specialization = input("Enter Specialization: ")

doctor = Doctor(
    doctor_id,
    doctor_name,
    specialization
)

print("\nEnter Appointment Details")
appointment_id = input("Enter Appointment ID: ")
date = input("Enter Appointment Date: ")

appointment = Appointment(
    appointment_id,
    patient,
    doctor,
    date
)

print("\n===== PATIENT DETAILS =====")
patient.display_details()

print("\n===== DOCTOR DETAILS =====")
doctor.display_details()

print("\n===== APPOINTMENT DETAILS =====")
appointment.display_details()
```

Output

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== HOSPITAL MANAGEMENT SYSTEM =====

Enter Patient Details
Enter Patient ID: 005
Enter Patient Name: MAX
Enter Patient Age: 50
Enter Disease: FLU

Enter Doctor Details
Enter Doctor ID: 01
Enter Doctor Name: jay
Enter Specialization: flu

Enter Appointment Details
Enter Appointment ID: 002
Enter Appointment Date: 01/01/2026

===== PATIENT DETAILS =====
Patient ID : 005
Name       : MAX
Age        : 50
Disease    : FLU

===== DOCTOR DETAILS =====
Doctor ID      : 01
Name           : jay
Specialization : flu

===== APPOINTMENT DETAILS =====
Appointment ID : 002
Date           : 01/01/2026
Patient        : MAX
Doctor         : jay
Specialization : flu
```

Practical 4: Online Shopping System

Description:

Develop classes for Customer, Product, Cart, and Order to simulate an online shopping platform.

Concepts Covered

- Objects
- Composition
- Classes

Input:

```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

class Product:

    def __init__(self, product_id, name, price):
        self.product_id = product_id
        self.name = name
        self.price = price

    def display_product(self):
        print("Product ID :", self.product_id)
        print("Name       :", self.name)
        print("Price      : ₹", self.price)


class Customer:

    def __init__(self, customer_id, name, email):
        self.customer_id = customer_id
        self.name = name
        self.email = email

    def display_customer(self):
        print("Customer ID :", self.customer_id)
        print("Name        :", self.name)
        print("Email       :", self.email)


class Cart:

    def __init__(self):
        self.products = []

    def add_product(self, product, quantity):
        self.products.append((product, quantity))
        print(f"{quantity} x {product.name} added to cart.")
```

```
    def display_cart(self):
        print("\n===== SHOPPING CART =====")

        if not self.products:
            print("Cart is empty.")
            return

        total = 0

        for product, quantity in self.products:
            item_total = product.price * quantity

            print("Product  :", product.name)
            print("Quantity :", quantity)
            print("Price    : ₹", product.price)
            print("Subtotal : ₹", item_total)
            print("-------------------------")

            total += item_total

        print("Total Cart Amount: ₹", total)

    def calculate_total(self):
        total = 0

        for product, quantity in self.products:
            total += product.price * quantity

        return total
```

```
class Order:

    def __init__(self, order_id, customer, cart):
        self.order_id = order_id
        self.customer = customer
        self.cart = cart

    def display_order(self):
        print("\n===== ORDER DETAILS =====")
        print("Order ID    :", self.order_id)
        print("Customer    :", self.customer.name)
        print("Email       :", self.customer.email)

        print("\nOrdered Products:")

        for product, quantity in self.cart.products:
            print(
                product.name,
                "x",
                quantity,
                "= ₹",
                product.price * quantity
            )

        print("\nTotal Amount: ₹", self.cart.calculate_total())
```

```
print("===== ONLINE SHOPPING SYSTEM =====")
print("\nEnter Customer Details")
customer_id = input("Enter Customer ID: ")
customer_name = input("Enter Customer Name: ")
email = input("Enter Email: ")

customer = Customer(
    customer_id,
    customer_name,
    email
)

product1 = Product("P101", "Laptop", 55000)
product2 = Product("P102", "Mouse", 800)
product3 = Product("P103", "Keyboard", 1500)

print("\n===== AVAILABLE PRODUCTS =====")

product1.display_product()
print("-------------------------")

product2.display_product()
print("-------------------------")

product3.display_product()

cart = Cart()

print("\n===== ADD PRODUCTS TO CART =====")

quantity1 = int(input("Enter quantity of Laptop: "))
if quantity1 > 0:
    cart.add_product(product1, quantity1)

quantity2 = int(input("Enter quantity of Mouse: "))
if quantity2 > 0:
    cart.add_product(product2, quantity2)

quantity3 = int(input("Enter quantity of Keyboard: "))
if quantity3 > 0:
    cart.add_product(product3, quantity3)
```

```
cart.display_cart()

order_id = input("\nEnter Order ID: ")

order = Order(
    order_id,
    customer,
    cart
)

order.display_order()
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== ONLINE SHOPPING SYSTEM =====

Enter Customer Details
Enter Customer ID: 001
Enter Customer Name: jay
Enter Email: jethavajay

===== AVAILABLE PRODUCTS =====
Product ID : P101
Name       : Laptop
Price      : ₹ 55000
----------------------
Product ID : P102
Name       : Mouse
Price      : ₹ 800
----------------------
Product ID : P103
Name       : Keyboard
Price      : ₹ 1500

===== ADD PRODUCTS TO CART =====
Enter quantity of Laptop: 2
2 x Laptop added to cart.
Enter quantity of Mouse: 2
2 x Mouse added to cart.
Enter quantity of Keyboard: 2
2 x Keyboard added to cart.

===== SHOPPING CART =====
Product  : Laptop
Quantity : 2
Price    : ₹ 55000
Subtotal : ₹ 110000
----------------------
Product  : Mouse
Quantity : 2
Price    : ₹ 800
Subtotal : ₹ 1600
----------------------
Product  : Keyboard
Quantity : 2
Price    : ₹ 1500
Subtotal : ₹ 3000
----------------------
Total Cart Amount: ₹ 114600

Enter Order ID: 005

===== ORDER DETAILS =====
Order ID : 005
Customer : jay
Email    : jethavajay

Ordered Products:
Laptop x 2 = ₹ 110000
Mouse x 2 = ₹ 1600
Keyboard x 2 = ₹ 3000

Total Amount: ₹ 114600
```

Practical 5: Vehicle Rental System

Description:

Implement classes for different vehicle types demonstrating inheritance and method overriding.

Concepts Covered

- Inheritance
- Method Overriding

Input:

```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

class Vehicle:

    def __init__(self, vehicle_id, brand, rent_per_day):
        self.vehicle_id = vehicle_id
        self.brand = brand
        self.rent_per_day = rent_per_day

    def calculate_rent(self, days):
        return self.rent_per_day * days

    def display_details(self):
        print("Vehicle ID   :", self.vehicle_id)
        print("Brand        :", self.brand)
        print("Rent per Day : ₹", self.rent_per_day)

class Car(Vehicle):

    def calculate_rent(self, days):
        rent = self.rent_per_day * days

        if days > 5:
            rent = rent * 0.90

        return rent

    def display_details(self):
        print("Vehicle Type : Car")
        super().display_details()

class Bike(Vehicle):

    def calculate_rent(self, days):
        rent = self.rent_per_day * days

        if days > 3:
            rent = rent * 0.95

        return rent

    def display_details(self):
        print("Vehicle Type : Bike")
        super().display_details()
```

```
class Bus(Vehicle):

    def calculate_rent(self, days):
        rent = self.rent_per_day * days
        rent = rent * 1.10

        return rent

    def display_details(self):
        print("Vehicle Type : Bus")
        super().display_details()

print("===== VEHICLE RENTAL SYSTEM =====")
print("\nSelect Vehicle Type")
print("1. Car")
print("2. Bike")
print("3. Bus")

choice = input("Enter your choice: ")

vehicle_id = input("Enter Vehicle ID: ")
brand = input("Enter Vehicle Brand: ")
rent_per_day = float(input("Enter Rent Per Day: ₹"))
days = int(input("Enter Number of Rental Days: "))

if choice == "1":
    vehicle = Car(vehicle_id, brand, rent_per_day)

elif choice == "2":
    vehicle = Bike(vehicle_id, brand, rent_per_day)

elif choice == "3":
    vehicle = Bus(vehicle_id, brand, rent_per_day)

else:
    print("Invalid vehicle type.")
    exit()

print("\n===== VEHICLE DETAILS =====")
vehicle.display_details()

total_rent = vehicle.calculate_rent(days)

print("Rental Days  :", days)
print("Total Rent   : ₹", total_rent)
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== VEHICLE RENTAL SYSTEM =====

Select Vehicle Type
1. Car
2. Bike
3. Bus
Enter your choice: 1
Enter Vehicle ID: 001
Enter Vehicle Brand: toyota
Enter Rent Per Day: ₹5000
Enter Number of Rental Days: 3

===== VEHICLE DETAILS =====
Vehicle Type : Car
Vehicle ID   : 001
Brand        : toyota
Rent per Day : ₹ 5000.0
Rental Days  : 3
Total Rent   : ₹ 15000.0
```

Practical 6: College Management System

Description:

Develop a complete college management system using multiple classes such as Student, Faculty, Course, and Department.

Concepts Covered

- OOP Design
- Information Hiding

Input:

```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

class Student:

    def __init__(self, student_id, name, semester):
        self.__student_id = student_id
        self.__name = name
        self.__semester = semester
        self.__courses = []

    def enroll_course(self, course):
        self.__courses.append(course)

    def display_details(self):
        print("Student ID :", self.__student_id)
        print("Name       :", self.__name)
        print("Semester   :", self.__semester)

        print("Courses:")
        if self.__courses:
            for course in self.__courses:
                print(" -", course.get_course_name())
        else:
            print(" No courses enrolled.")

class Faculty:

    def __init__(self, faculty_id, name, designation):
        self.__faculty_id = faculty_id
        self.__name = name
        self.__designation = designation

    def get_name(self):
        return self.__name

    def display_details(self):
        print("Faculty ID  :", self.__faculty_id)
        print("Name        :", self.__name)
        print("Designation :", self.__designation)
```

```python
class Course:

    def __init__(self, course_id, course_name, credits):
        self.__course_id = course_id
        self.__course_name = course_name
        self.__credits = credits
        self.__faculty = None

    def get_course_name(self):
        return self.__course_name

    def assign_faculty(self, faculty):
        self.__faculty = faculty

    def display_details(self):
        print("Course ID   :", self.__course_id)
        print("Course Name :", self.__course_name)
        print("Credits     :", self.__credits)

        if self.__faculty:
            print("Faculty     :", self.__faculty.get_name())
        else:
            print("Faculty     : Not Assigned")
```

```python
class Department:

    def __init__(self, department_id, name):
        self.__department_id = department_id
        self.__name = name
        self.__students = []
        self.__faculty = []
        self.__courses = []

    def add_student(self, student):
        self.__students.append(student)

    def add_faculty(self, faculty):
        self.__faculty.append(faculty)

    def add_course(self, course):
        self.__courses.append(course)

    def display_department(self):
        print("\n===== DEPARTMENT DETAILS =====")
        print("Department ID   :", self.__department_id)
        print("Department Name :", self.__name)

        print("\n--- Students ---")
        if self.__students:
            for student in self.__students:
                student.display_details()
                print("--------------------")
        else:
            print("No students.")
```

```
        print("\n--- Faculty ---")
        if self.__faculty:
            for faculty in self.__faculty:
                faculty.display_details()
                print("--------------------")
        else:
            print("No faculty.")

        print("\n--- Courses ---")
        if self.__courses:
            for course in self.__courses:
                course.display_details()
                print("--------------------")
        else:
            print("No courses.")

print("===== COLLEGE MANAGEMENT SYSTEM =====")

department = Department("D01", "Computer Engineering")

student1 = Student("S101", "Rahul", 5)
student2 = Student("S102", "Priya", 5)

faculty1 = Faculty("F101", "Dr. Patel", "Professor")
faculty2 = Faculty("F102", "Dr. Shah", "Assistant Professor")

course1 = Course("C101", "Python Programming", 4)
course2 = Course("C102", "Data Structures", 4)

course1.assign_faculty(faculty1)
course2.assign_faculty(faculty2)

student1.enroll_course(course1)
student1.enroll_course(course2)

student2.enroll_course(course1)

department.add_student(student1)
department.add_student(student2)

department.add_faculty(faculty1)
department.add_faculty(faculty2)

department.add_course(course1)
department.add_course(course2)

department.display_department()
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== COLLEGE MANAGEMENT SYSTEM =====

===== DEPARTMENT DETAILS =====
Department ID   : D01
Department Name : Computer Engineering

--- Students ---
Student ID : S101
Name       : Rahul
Semester   : 5
Courses:
 - Python Programming
 - Data Structures
------------------
Student ID : S102
Name       : Priya
Semester   : 5
Courses:
 - Python Programming
------------------

--- Faculty ---
Faculty ID  : F101
Name        : Dr. Patel
Designation : Professor
------------------
Faculty ID  : F102
Name        : Dr. Shah
Designation : Assistant Professor
------------------

--- Courses ---
Course ID   : C101
Course Name : Python Programming
Credits     : 4
Faculty     : Dr. Patel
------------------
Course ID   : C102
Course Name : Data Structures
Credits     : 4
Faculty     : Dr. Shah
------------------
```

Unit 5: Data Science and Data Visualization

Practical 1: Student Performance Analysis using Pandas

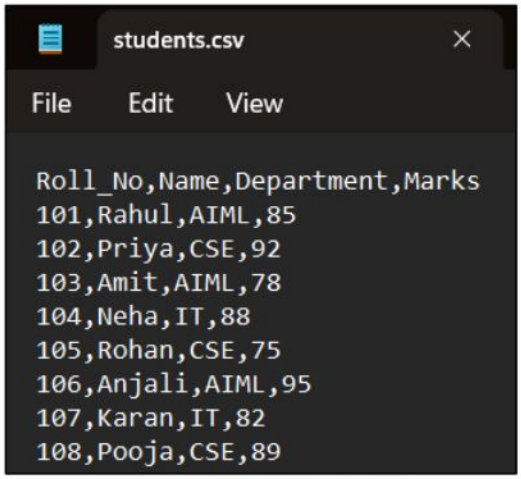
Description:

Import student marks from a CSV file into a DataFrame. Calculate average marks, highest marks, and generate department-wise statistics.

Concepts Covered

- Pandas
- DataFrames

Input:




```python
print("Name             :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import pandas as pd

df = pd.read_csv("students.csv")

print("===== STUDENT DATA =====")
print(df)

average_marks = df["Marks"].mean()
highest_marks = df["Marks"].max()
top_student = df.loc[df["Marks"].idxmax()]

department_stats = df.groupby("Department")["Marks"].agg(
    ["count", "mean", "max", "min"]
)

print("\n===== OVERALL STATISTICS =====")
print("Average Marks :", round(average_marks, 2))
print("Highest Marks :", highest_marks)

print("\n===== TOP PERFORMING STUDENT =====")
print("Roll No     :", top_student["Roll_No"])
print("Name        :", top_student["Name"])
print("Department  :", top_student["Department"])
print("Marks       :", top_student["Marks"])

print("\n===== DEPARTMENT-WISE STATISTICS =====")
print(department_stats)
```

Output:

```
===== STUDENT DATA =====
   Roll_No    Name Department  Marks
0      101   Rahul       AIML     85
1      102   Priya        CSE     92
2      103    Amit       AIML     78
3      104    Neha         IT     88
4      105   Rohan        CSE     75
5      106  Anjali       AIML     95
6      107   Karan         IT     82
7      108   Pooja        CSE     89

===== OVERALL STATISTICS =====
Average Marks : 85.5
Highest Marks : 95

===== TOP PERFORMING STUDENT =====
Roll No     : 106
Name        : Anjali
Department  : AIML
Marks       : 95

===== DEPARTMENT-WISE STATISTICS =====
            count       mean  max  min
Department
AIML            3  86.000000   95   78
CSE             3  85.333333   92   75
IT              2  85.000000   88   82
```

Practical 2: COVID-19 Data Visualization

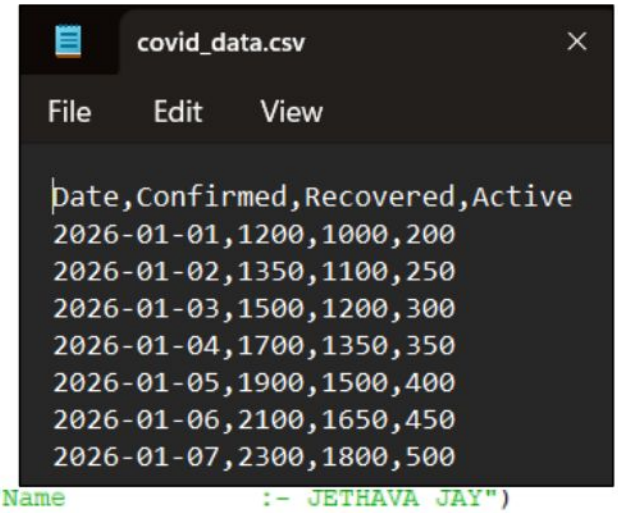
Description:

Analyze COVID-19 data stored in a CSV file and generate bar charts, pie charts, and line graphs for confirmed, recovered, and active cases.

Concepts Covered

- Pandas
- Matplotlib

Input:




```python
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("covid_data.csv")

df["Date"] = pd.to_datetime(df["Date"])

print("===== COVID-19 DATA =====")
print(df)

print("\n===== TOTAL CASES =====")
print("Total Confirmed :", df["Confirmed"].sum())
print("Total Recovered :", df["Recovered"].sum())
print("Total Active    :", df["Active"].sum())

# 1. BAR CHART
plt.figure(figsize=(8, 5))
categories = ["Confirmed", "Recovered", "Active"]
values = [
    df["Confirmed"].sum(),
    df["Recovered"].sum(),
    df["Active"].sum()
]
plt.bar(categories, values)
plt.title("COVID-19 Cases - Bar Chart")
plt.xlabel("Case Type")
plt.ylabel("Number of Cases")
plt.show()
```

```
# 2. PIE CHART
plt.figure(figsize=(7, 7))
plt.pie(
    values,
    labels=categories,
    autopct="%1.1f%%"
)
plt.title("COVID-19 Cases Distribution")
plt.show()

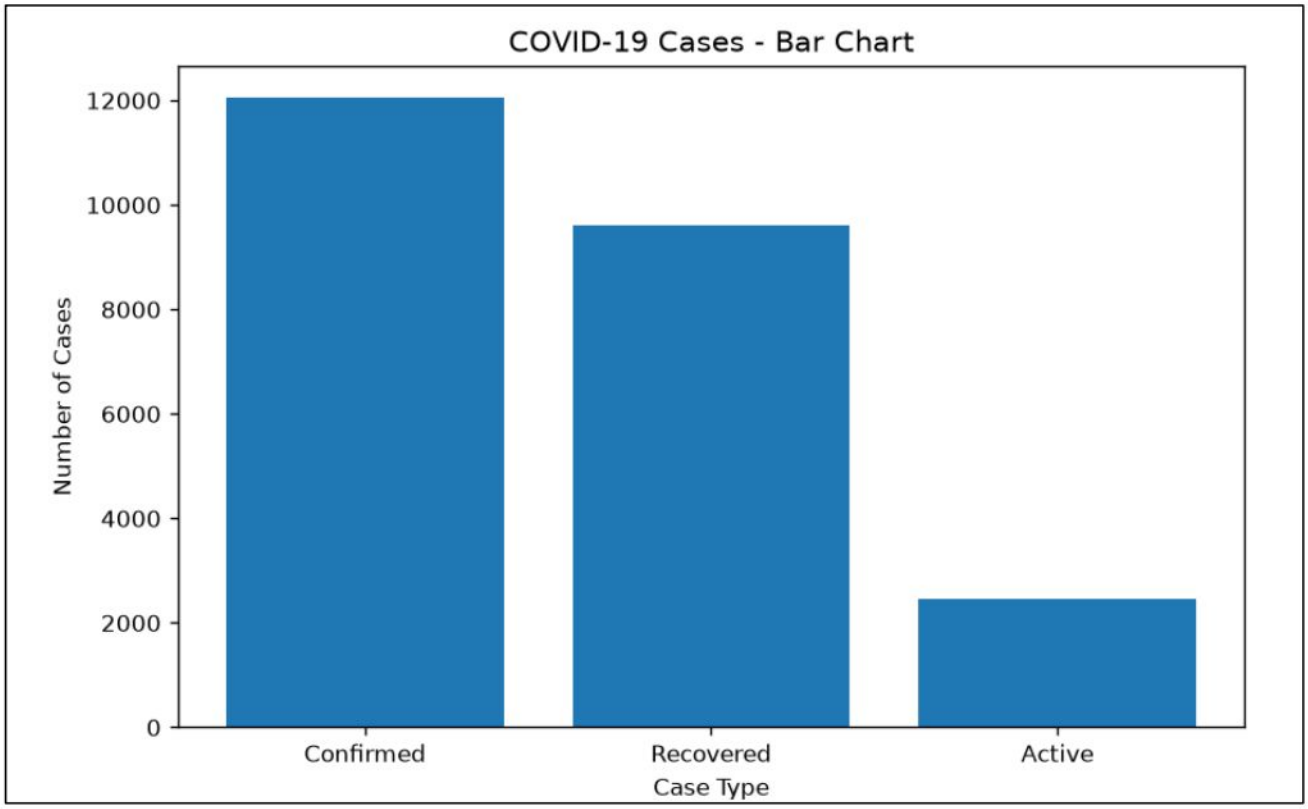
# 3. LINE GRAPH
plt.figure(figsize=(9, 5))
plt.plot(
    df["Date"],
    df["Confirmed"],
    marker="o",
    label="Confirmed"
)
plt.plot(
    df["Date"],
    df["Recovered"],
    marker="o",
    label="Recovered"
)
plt.plot(
    df["Date"],
    df["Active"],
    marker="o",
    label="Active"
)
plt.title("COVID-19 Cases Over Time")
plt.xlabel("Date")
plt.ylabel("Number of Cases")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

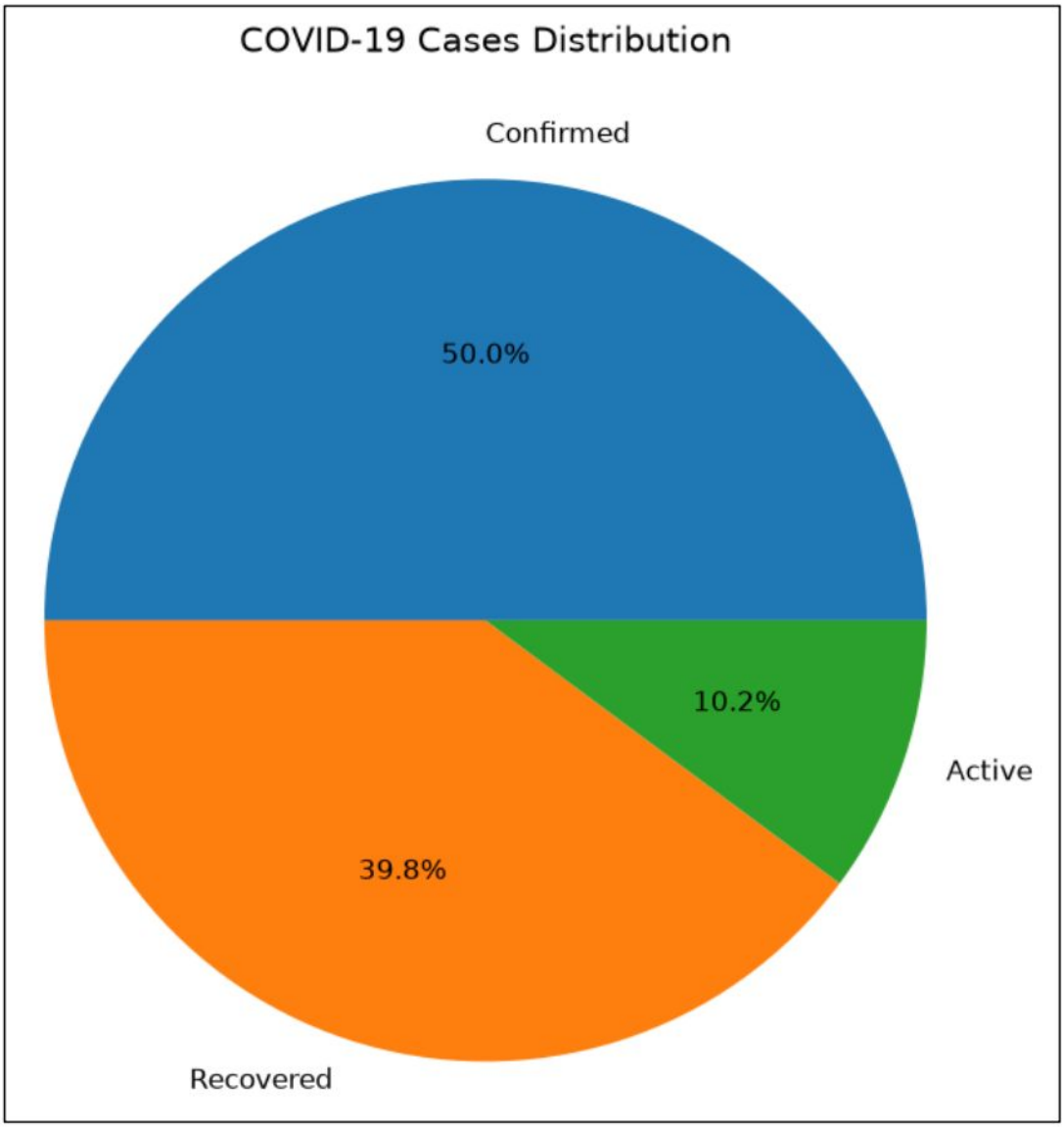
Output:

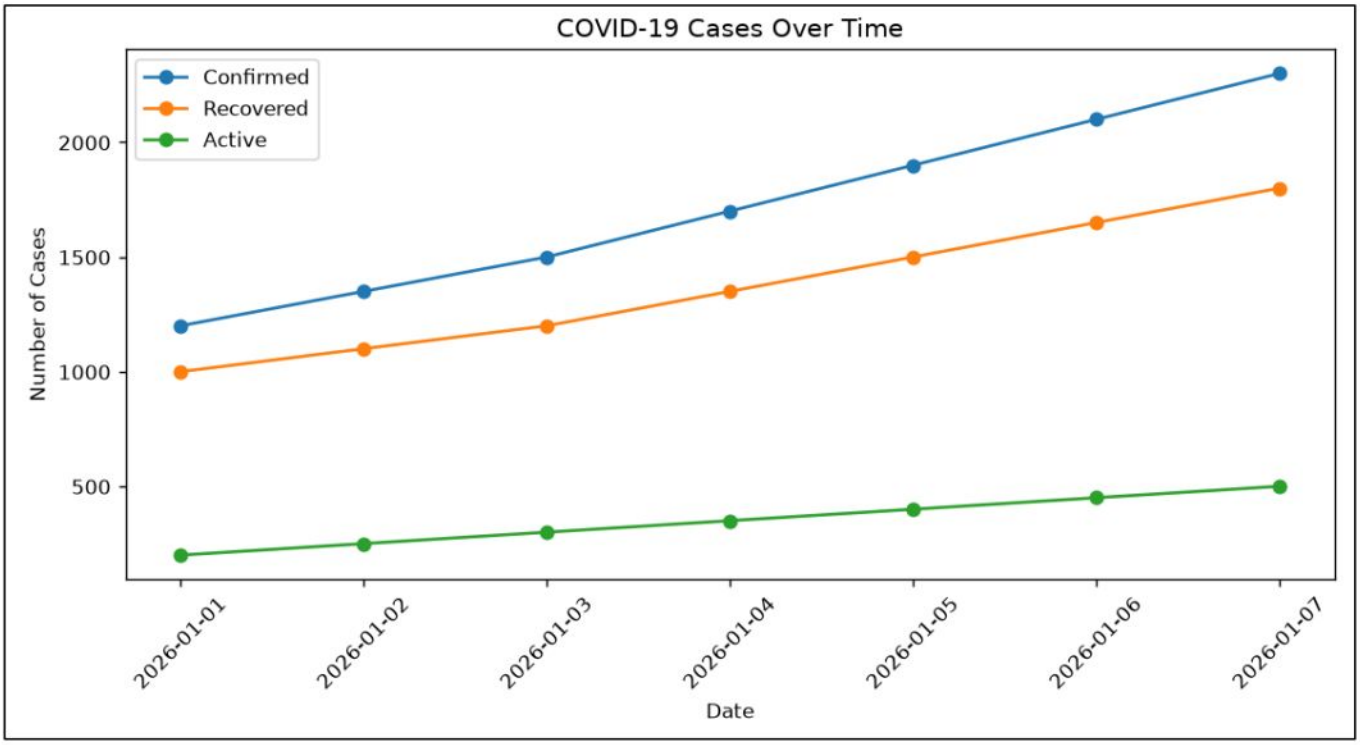
```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== COVID-19 DATA =====
        Date  Confirmed  Recovered  Active
0 2026-01-01       1200       1000     200
1 2026-01-02       1350       1100     250
2 2026-01-03       1500       1200     300
3 2026-01-04       1700       1350     350
4 2026-01-05       1900       1500     400
5 2026-01-06       2100       1650     450
6 2026-01-07       2300       1800     500

===== TOTAL CASES =====
Total Confirmed : 12050
Total Recovered : 9600
Total Active    : 2450
```


COVID-19 Cases - Bar Chart
Number of Cases
12000
10000
8000
6000
4000
2000
0
Confirmed
Recovered
Active
Case Type


COVID-19 Cases Distribution
Confirmed
50.0%
10.2%
Active
39.8%
Recovered


COVID-19 Cases Over Time
Confirmed
Recovered
Active
Number of Cases
2000
1500
1000
500
2026-01-01
2026-01-02
2026-01-03
2026-01-04
2026-01-05
2026-01-06
2026-01-07
Date

Practical 3: Sales Dashboard

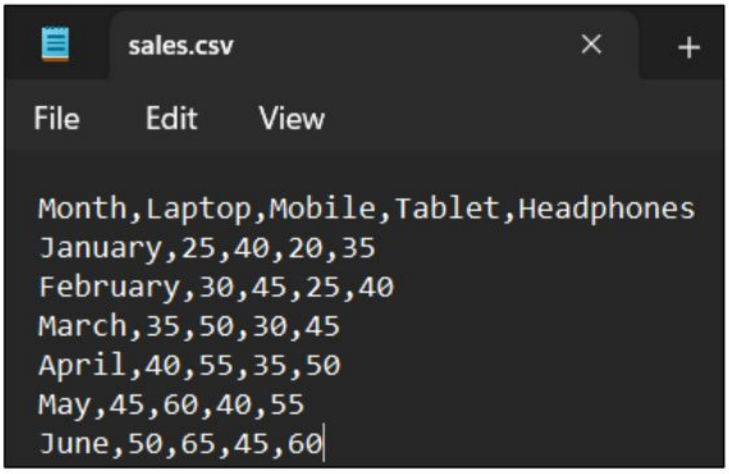
Description:

Create a dashboard that visualizes monthly product sales using multiple graph types.

Concepts Covered

- DataFrames
- Graph Plotting

Input:




```
print("Name             :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("sales.csv")

print("===== SALES DATA =====")
print(df)

product_totals = df[
    ["Laptop", "Mobile", "Tablet", "Headphones"]
].sum()

print("\n===== TOTAL PRODUCT SALES =====")
print(product_totals)

df["Total Sales"] = df[
    ["Laptop", "Mobile", "Tablet", "Headphones"]
].sum(axis=1)

print("\n===== MONTHLY TOTAL SALES =====")
print(df[["Month", "Total Sales"]])
```

```
# 1. LINE GRAPH - Monthly Sales
plt.figure(figsize=(9, 5))
plt.plot(
    df["Month"],
    df["Laptop"],
    marker="o",
    label="Laptop"
)
plt.plot(
    df["Month"],
    df["Mobile"],
    marker="o",
    label="Mobile"
)
plt.plot(
    df["Month"],
    df["Tablet"],
    marker="o",
    label="Tablet"
)
plt.plot(
    df["Month"],
    df["Headphones"],
    marker="o",
    label="Headphones"
)
plt.title("Monthly Product Sales")
plt.xlabel("Month")
plt.ylabel("Number of Units Sold")
plt.legend()
plt.grid(True)
plt.show()
```

```
# 2. BAR CHART - Product-wise Sales
plt.figure(figsize=(8, 5))
plt.bar(
    product_totals.index,
    product_totals.values
)
plt.title("Total Product Sales")
plt.xlabel("Products")
plt.ylabel("Total Units Sold")
plt.show()

# 3. PIE CHART - Sales Distribution
plt.figure(figsize=(7, 7))
plt.pie(
    product_totals.values,
    labels=product_totals.index,
    autopct="%1.1f%%"
)
plt.title("Product Sales Distribution")
plt.show()

# 4. BAR CHART - Monthly Total Sales
plt.figure(figsize=(9, 5))
plt.bar(
    df["Month"],
    df["Total Sales"]
)
plt.title("Monthly Total Sales")
plt.xlabel("Month")
plt.ylabel("Total Units Sold")
plt.show()
```

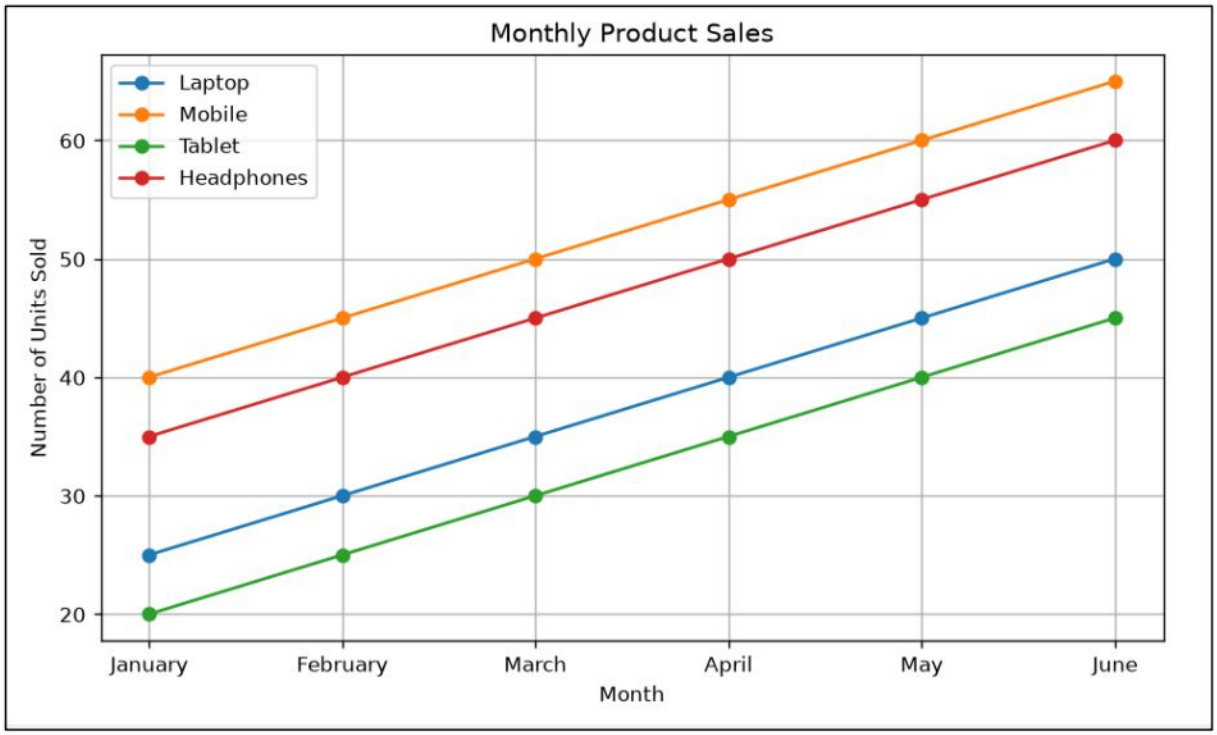
Output:

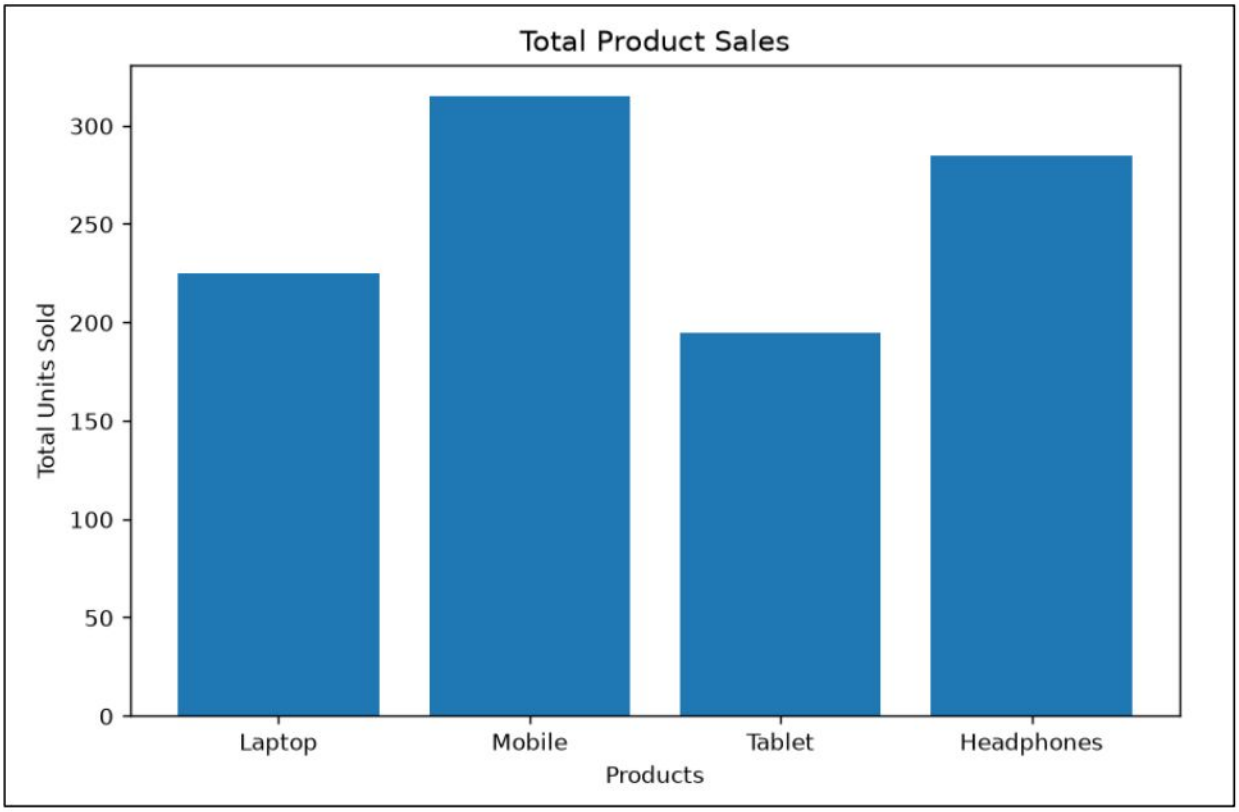
```
======================== RESTART: D:/New folder/PYTHON.]
Name              :- JETHAVA JAY
Enrollment No. :- 250283152004

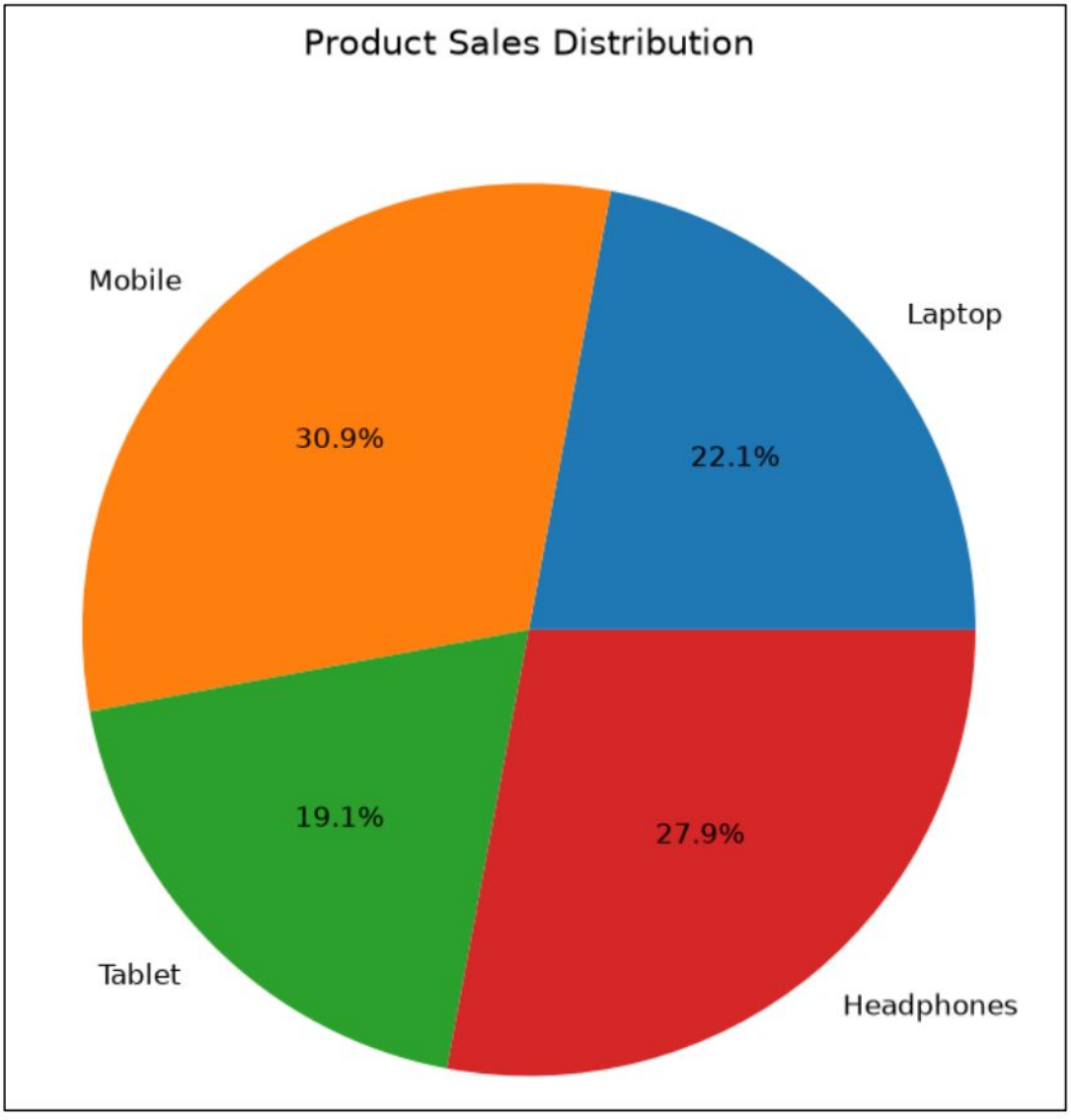
===== SALES DATA =====
      Month  Laptop  Mobile  Tablet  Headphones
0   January      25      40      20          35
1  February      30      45      25          40
2     March      35      50      30          45
3     April      40      55      35          50
4       May      45      60      40          55
5      June      50      65      45          60

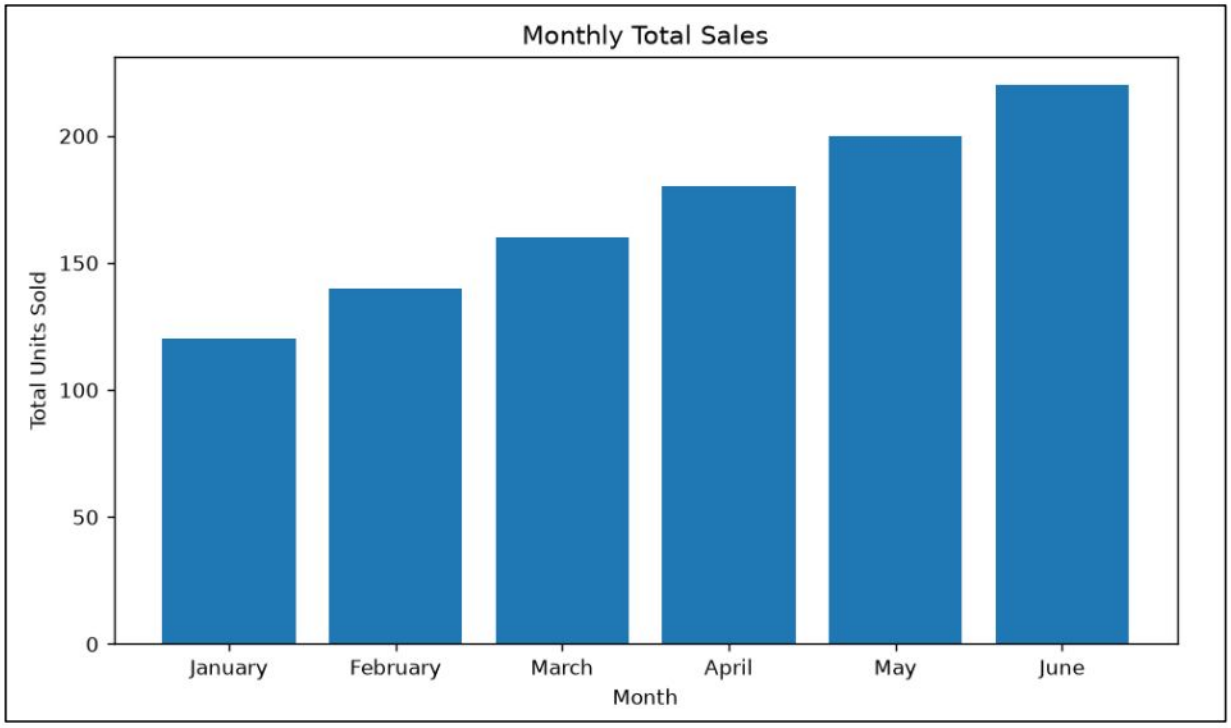
===== TOTAL PRODUCT SALES =====
Laptop        225
Mobile        315
Tablet        195
Headphones    285
dtype: int64

===== MONTHLY TOTAL SALES =====
      Month  Total Sales
0   January          120
1  February          140
2     March          160
3     April          180
4       May          200
5      June          220
```




Total Product Sales
Total Units Sold
300
250
200
150
100
50
0
Laptop
Mobile
Tablet
Headphones
Products


Product Sales Distribution
Mobile
30.9%
Laptop
22.1%
19.1%
Tablet
27.9%
Headphones


Monthly Total Sales
Total Units Sold
0
50
100
150
200
January
February
March
April
May
June
Month

Practical 4: IPL Statistics Analyzer

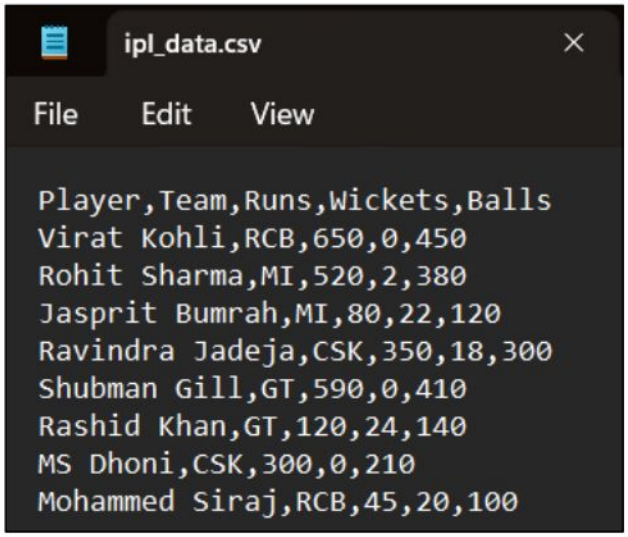
Description:

Analyze IPL player performance and visualize runs, wickets, strike rates, and team-wise statistics.

Concepts Covered

- Data Analysis
- Charts

Input:




```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("ipl_data.csv")

print("===== IPL PLAYER DATA =====")
print(df)

# Calculate Strike Rate
df["Strike_Rate"] = (df["Runs"] / df["Balls"]) * 100
df["Strike_Rate"] = df["Strike_Rate"].round(2)

# Player with Highest Runs
top_batsman = df.loc[df["Runs"].idxmax()]

print("\n===== TOP RUN SCORER =====")
print("Player      :", top_batsman["Player"])
print("Team        :", top_batsman["Team"])
print("Runs        :", top_batsman["Runs"])
print("Strike Rate :", top_batsman["Strike_Rate"])

# Player with Most Wickets
top_bowler = df.loc[df["Wickets"].idxmax()]

print("\n===== TOP WICKET TAKER =====")
print("Player      :", top_bowler["Player"])
print("Team        :", top_bowler["Team"])
print("Wickets     :", top_bowler["Wickets"])
```

```
# Team-wise Statistics
team_stats = df.groupby("Team").agg(
    Total_Runs=("Runs", "sum"),
    Total_Wickets=("Wickets", "sum")
)

print("\n===== TEAM-WISE STATISTICS =====")
print(team_stats)

# 1. BAR CHART - Player Runs
plt.figure(figsize=(10, 5))
plt.bar(
    df["Player"],
    df["Runs"]
)
plt.title("IPL Player Runs")
plt.xlabel("Players")
plt.ylabel("Runs")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 2. BAR CHART - Player Wickets
plt.figure(figsize=(10, 5))
plt.bar(
    df["Player"],
    df["Wickets"]
)
plt.title("IPL Player Wickets")
plt.xlabel("Players")
plt.ylabel("Wickets")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# 3. LINE GRAPH - Strike Rate
plt.figure(figsize=(10, 5))
plt.plot(
    df["Player"],
    df["Strike_Rate"],
    marker="o"
)
plt.title("IPL Player Strike Rate")
plt.xlabel("Players")
plt.ylabel("Strike Rate")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 4. TEAM-WISE RUNS
plt.figure(figsize=(8, 5))
plt.bar(
    team_stats.index,
    team_stats["Total_Runs"]
)
plt.title("Team-wise Total Runs")
plt.xlabel("Team")
plt.ylabel("Total Runs")
plt.show()
```

```
# 5. TEAM-WISE WICKETS
plt.figure(figsize=(8, 5))
plt.bar(
    team_stats.index,
    team_stats["Total_Wickets"]
)
plt.title("Team-wise Total Wickets")
plt.xlabel("Team")
plt.ylabel("Total Wickets")
plt.show()
```

Output:

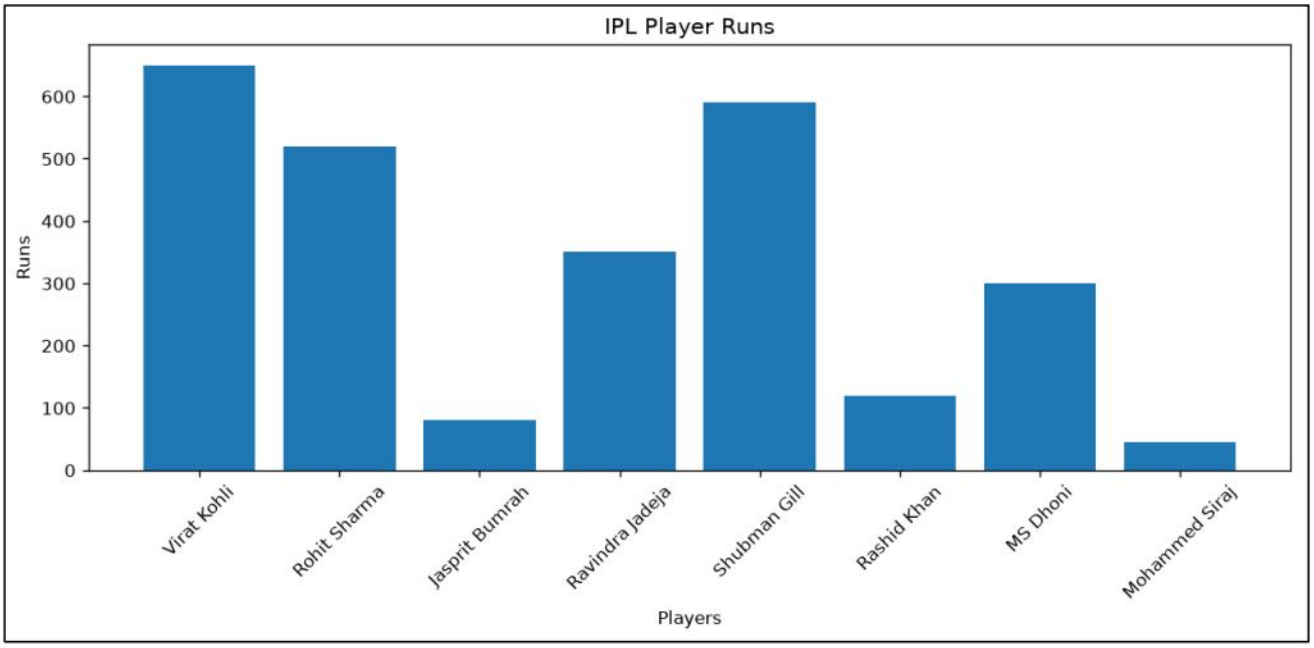
```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

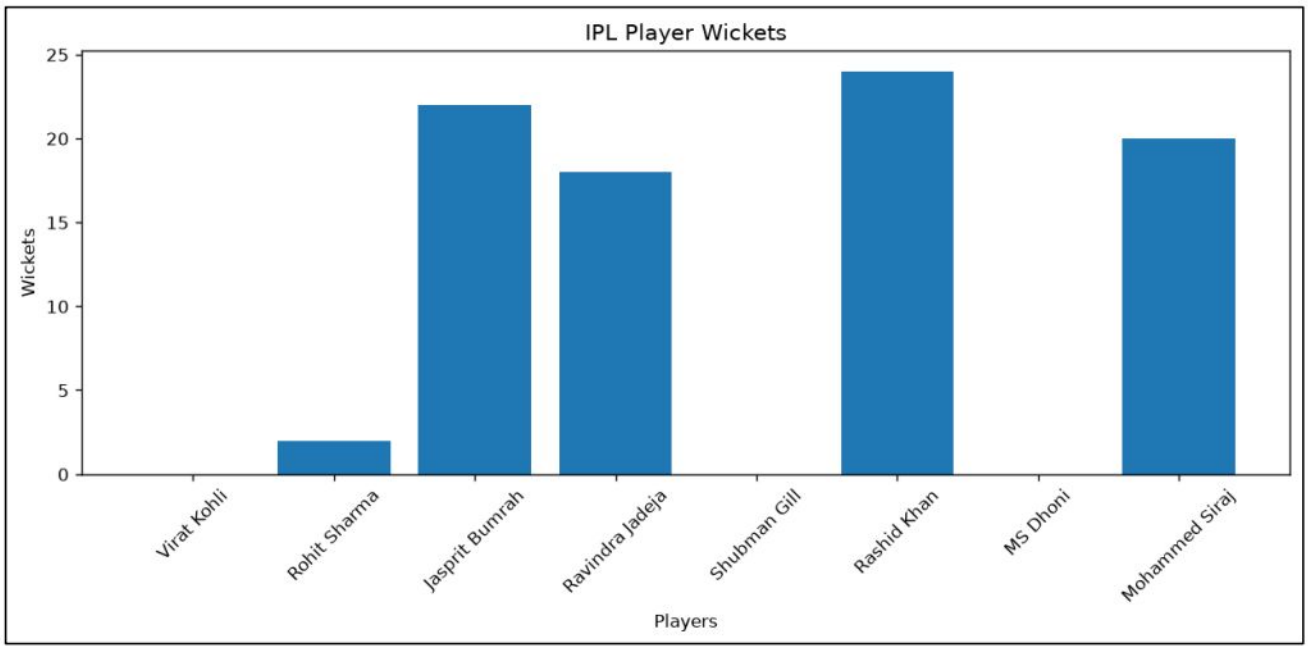
===== IPL PLAYER DATA =====
            Player Team  Runs  Wickets  Balls
0      Virat Kohli  RCB   650        0    450
1     Rohit Sharma   MI   520        2    380
2   Jasprit Bumrah   MI    80       22    120
3  Ravindra Jadeja  CSK   350       18    300
4     Shubman Gill   GT   590        0    410
5      Rashid Khan   GT   120       24    140
6         MS Dhoni  CSK   300        0    210
7   Mohammed Siraj  RCB    45       20    100

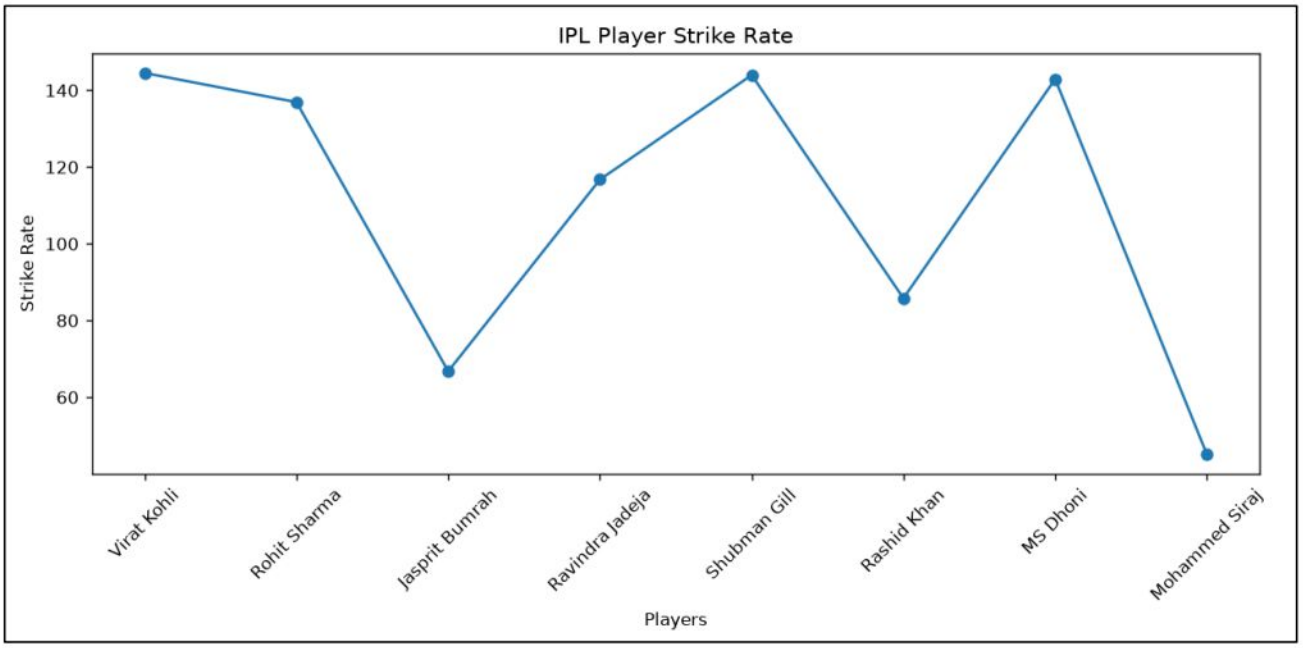
===== TOP RUN SCORER =====
Player      : Virat Kohli
Team        : RCB
Runs        : 650
Strike Rate : 144.44

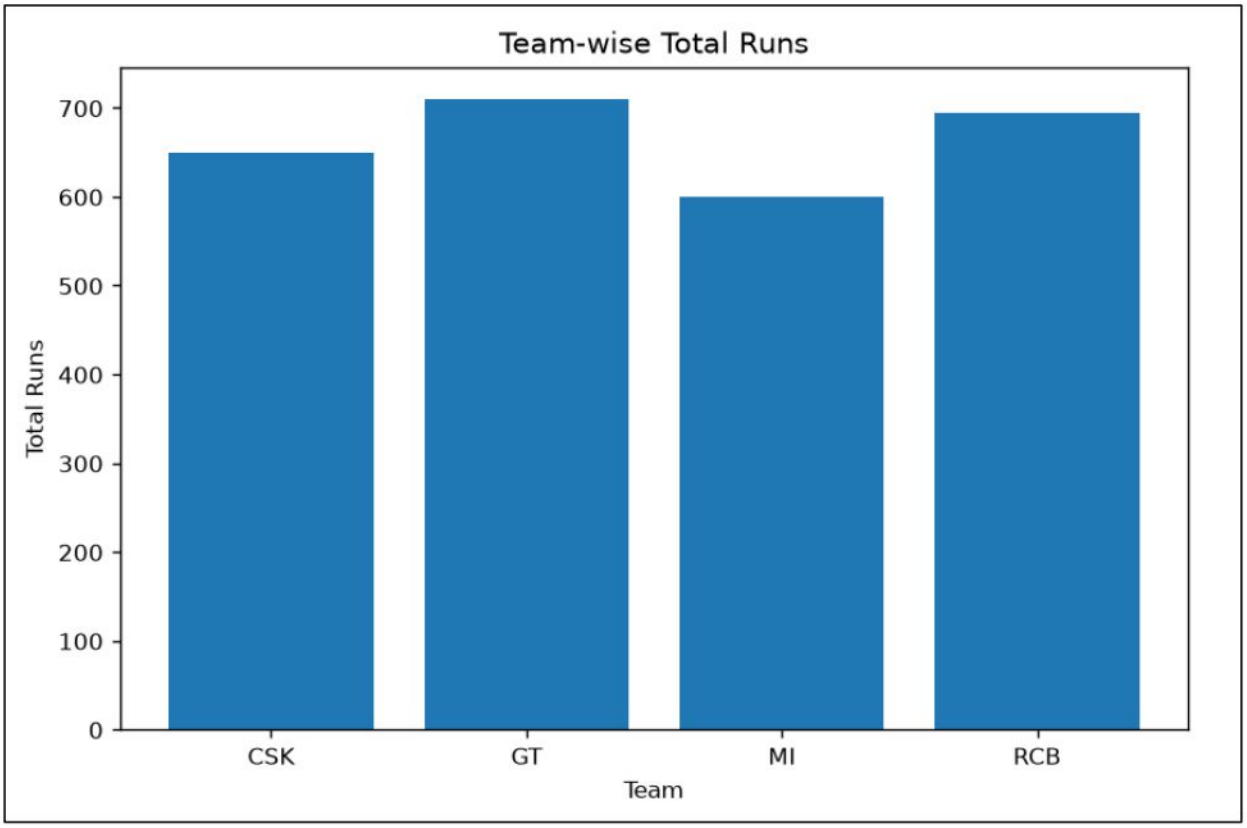
===== TOP WICKET TAKER =====
Player      : Rashid Khan
Team        : GT
Wickets     : 24

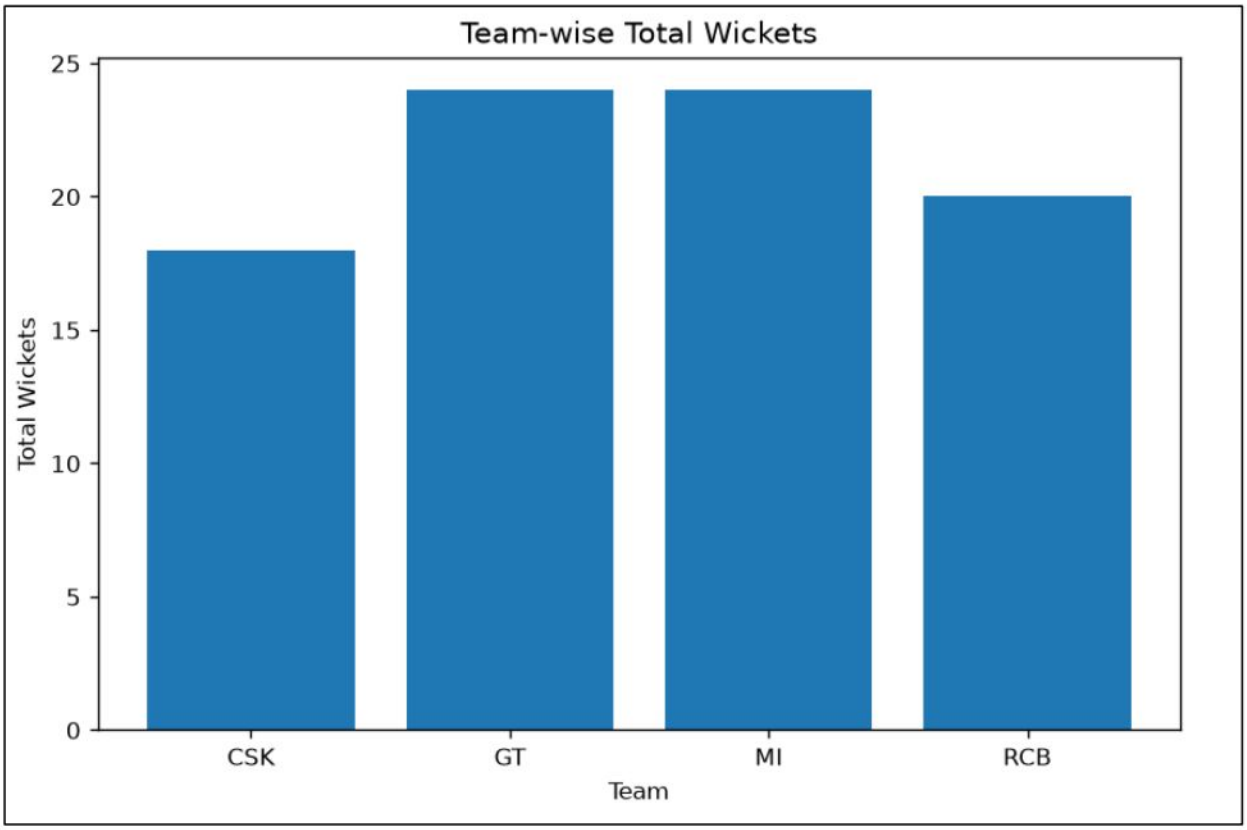
===== TEAM-WISE STATISTICS =====
      Total_Runs  Total_Wickets
Team
CSK          650             18
GT           710             24
MI           600             24
RCB          695             20
```


IPL Player Runs
Runs
0
100
200
300
400
500
600
Virat Kohli
Rohit Sharma
Jasprit Bumrah
Ravindra Jadeja
Shubman Gill
Rashid Khan
MS Dhoni
Mohammed Siraj
Players


IPL Player Wickets
Wickets
0
5
10
15
20
25
Virat Kohli
Rohit Sharma
Jasprit Bumrah
Ravindra Jadeja
Shubman Gill
Rashid Khan
MS Dhoni
Mohammed Siraj
Players


IPL Player Strike Rate
Strike Rate
60
80
100
120
140
Virat Kohli
Rohit Sharma
Jasprit Bumrah
Ravindra Jadeja
Shubman Gill
Rashid Khan
MS Dhoni
Mohammed Siraj
Players


Team-wise Total Runs
Total Runs
0
100
200
300
400
500
600
700
CSK
GT
MI
RCB
Team


Team-wise Total Wickets
Total Wickets
0
5
10
15
20
25
CSK
GT
MI
RCB
Team

Practical 5: Weather Data Visualization

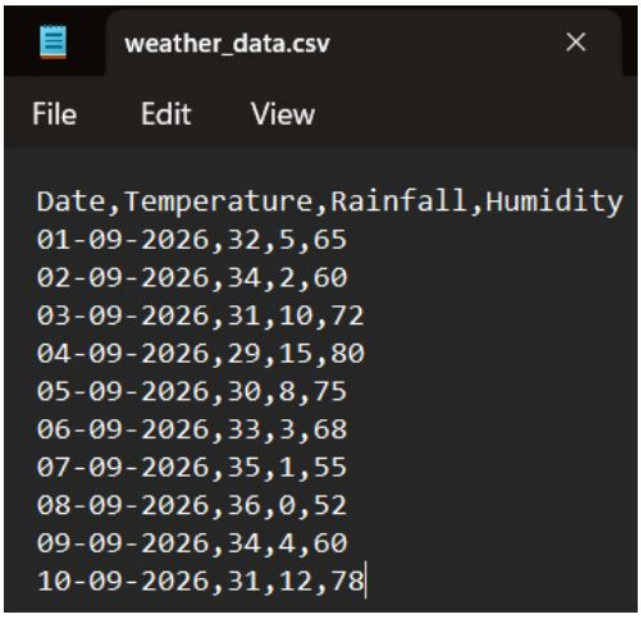
Description:

Visualize daily temperature, rainfall, and humidity data using line charts, histograms, and bar charts.

Concepts Covered

- Plotting
- Histograms

Input:




```
print("Name             :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("weather_data.csv")

df["Date"] = pd.to_datetime(
    df["Date"],
    format="%d-%m-%Y"
)

print("===== WEATHER DATA =====")
print(df)

# Display Basic Statistics
print("\n===== WEATHER STATISTICS =====")
print("Average Temperature :",
      round(df["Temperature"].mean(), 2), "°C")
print("Total Rainfall       :",
      df["Rainfall"].sum(), "mm")
print("Average Humidity    :",
      round(df["Humidity"].mean(), 2), "%")
```

```python
# 1. LINE CHART - Temperature
plt.figure(figsize=(9, 5))
plt.plot(
    df["Date"],
    df["Temperature"],
    marker="o"
)
plt.title("Daily Temperature")
plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 2. LINE CHART - Rainfall
plt.figure(figsize=(9, 5))
plt.plot(
    df["Date"],
    df["Rainfall"],
    marker="o"
)
plt.title("Daily Rainfall")
plt.xlabel("Date")
plt.ylabel("Rainfall (mm)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 3. LINE CHART - Humidity
plt.figure(figsize=(9, 5))
plt.plot(
    df["Date"],
    df["Humidity"],
    marker="o"
)
plt.title("Daily Humidity")
plt.xlabel("Date")
plt.ylabel("Humidity (%)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```python
# 4. HISTOGRAM - Temperature
plt.figure(figsize=(8, 5))
plt.hist(
    df["Temperature"],
    bins=5,
    edgecolor="black"
)
plt.title("Temperature Distribution")
plt.xlabel("Temperature (°C)")
plt.ylabel("Frequency")
plt.show()

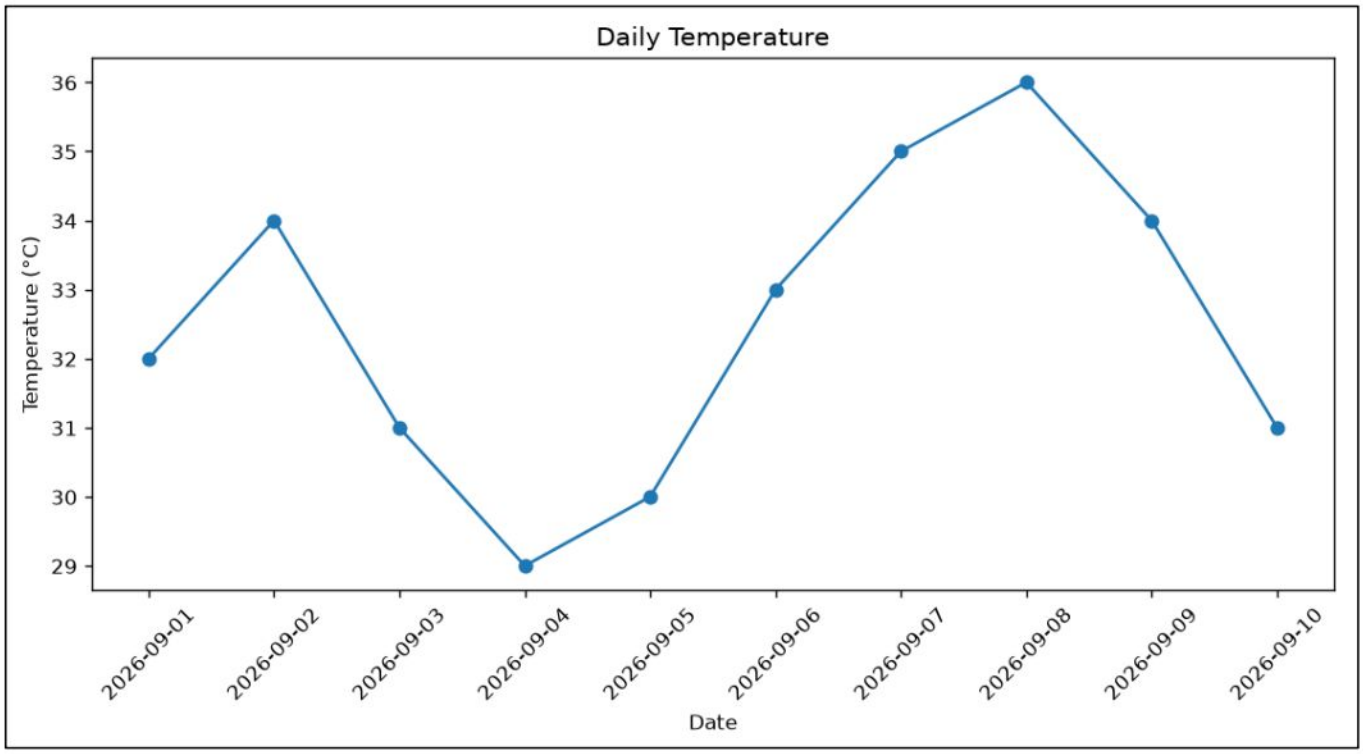
# 5. BAR CHART - Rainfall
plt.figure(figsize=(9, 5))
plt.bar(
    df["Date"].dt.strftime("%d-%m"),
    df["Rainfall"]
)
plt.title("Daily Rainfall")
plt.xlabel("Date")
plt.ylabel("Rainfall (mm)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

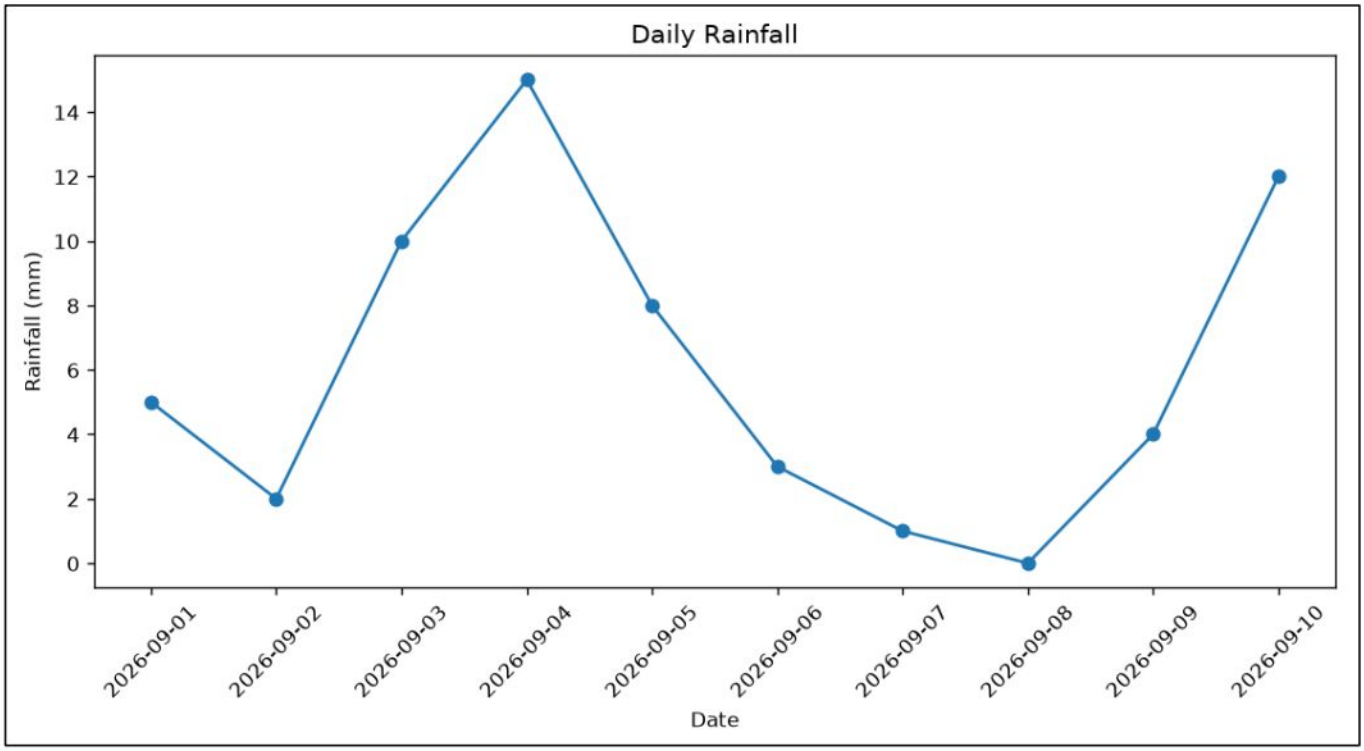
Output:

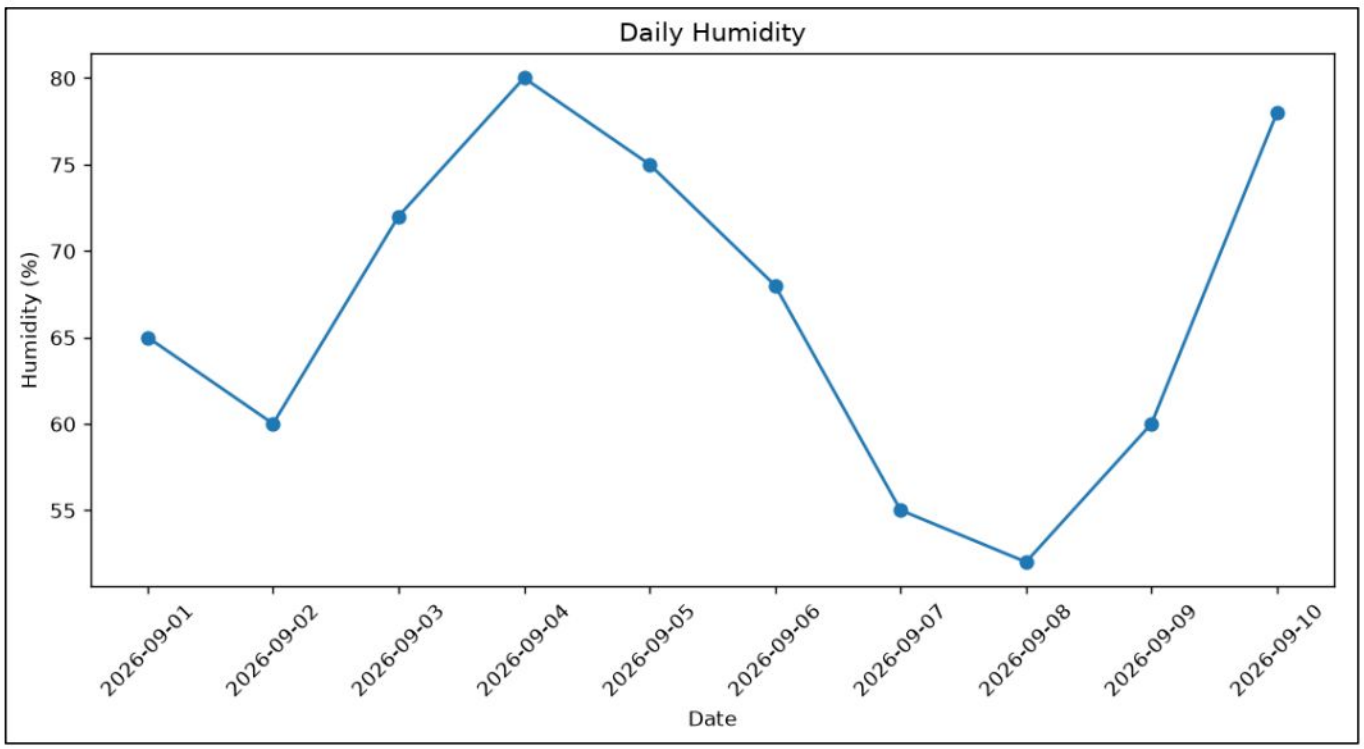
```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

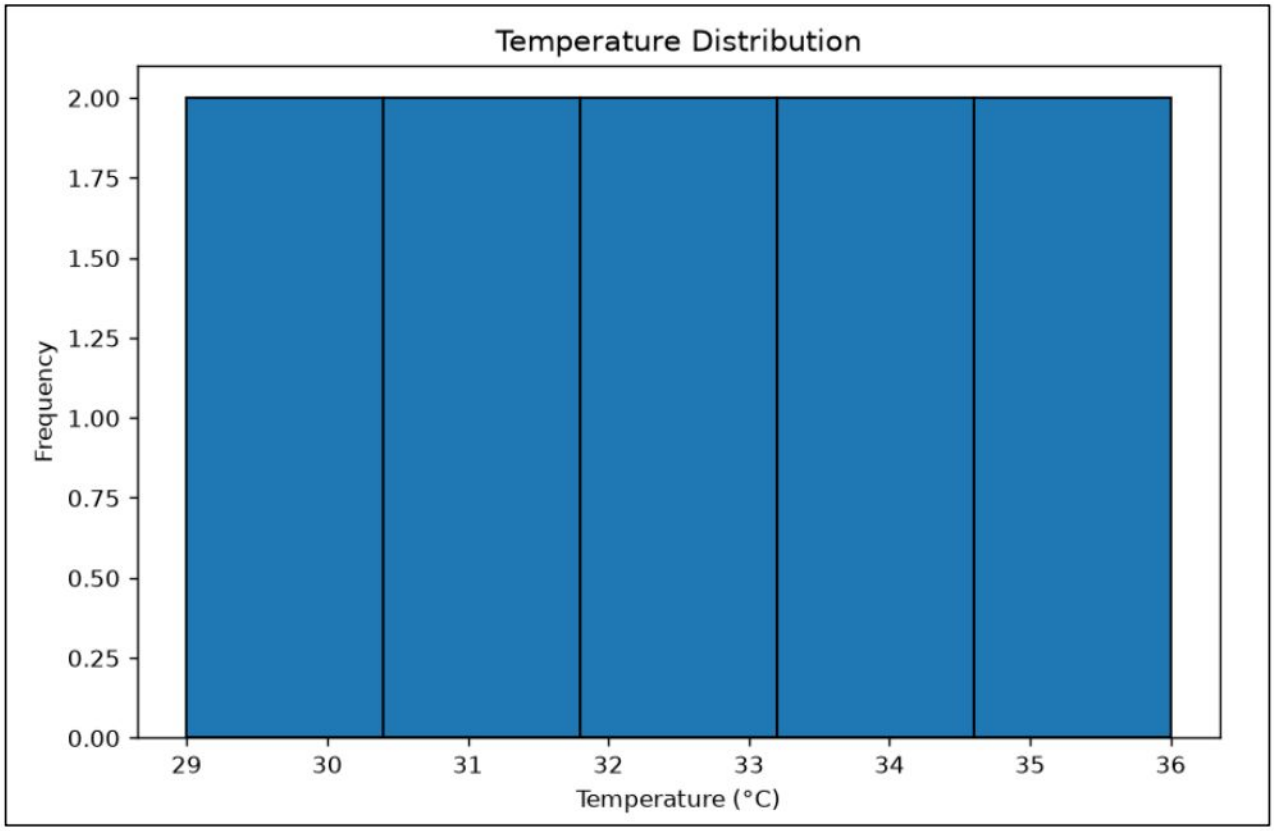
===== WEATHER DATA =====
        Date  Temperature  Rainfall  Humidity
0 2026-09-01           32         5        65
1 2026-09-02           34         2        60
2 2026-09-03           31        10        72
3 2026-09-04           29        15        80
4 2026-09-05           30         8        75
5 2026-09-06           33         3        68
6 2026-09-07           35         1        55
7 2026-09-08           36         0        52
8 2026-09-09           34         4        60
9 2026-09-10           31        12        78

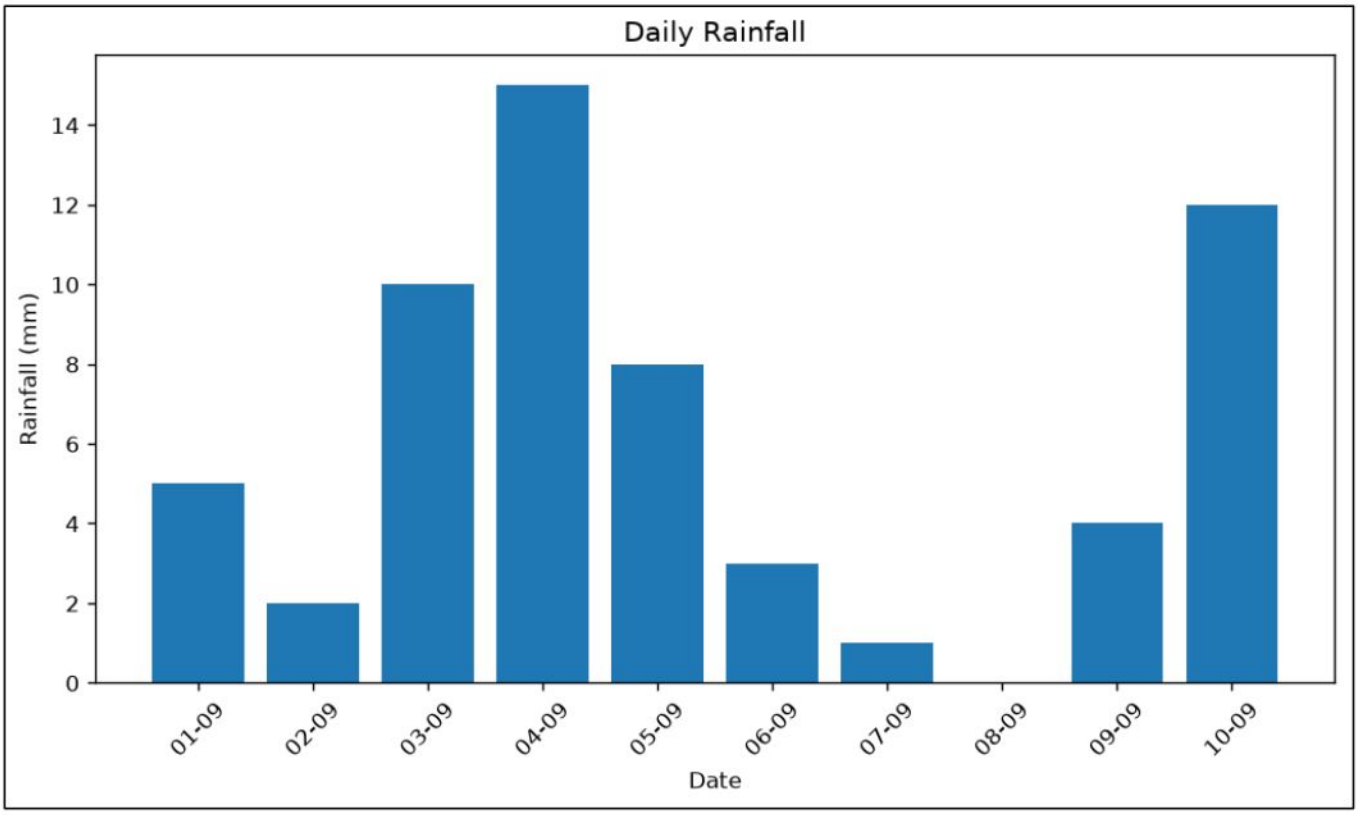
===== WEATHER STATISTICS =====
Average Temperature : 32.5 °C
Total Rainfall      : 60 mm
Average Humidity    : 66.5 %
```




Daily Rainfall
Rainfall (mm)
0
2
4
6
8
10
12
14
2026-09-01
2026-09-02
2026-09-03
2026-09-04
2026-09-05
2026-09-06
2026-09-07
2026-09-08
2026-09-09
2026-09-10
Date


Daily Humidity
Humidity (%)
55
60
65
70
75
80
2026-09-01
2026-09-02
2026-09-03
2026-09-04
2026-09-05
2026-09-06
2026-09-07
2026-09-08
2026-09-09
2026-09-10
Date


Temperature Distribution
Frequency
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
29
30
31
32
33
34
35
36
Temperature (°C)


Daily Rainfall
Rainfall (mm)
14
12
10
8
6
4
2
0
01-09
02-09
03-09
04-09
05-09
06-09
07-09
08-09
09-09
10-09
Date

Practical 6: Stock Market Trend Analysis

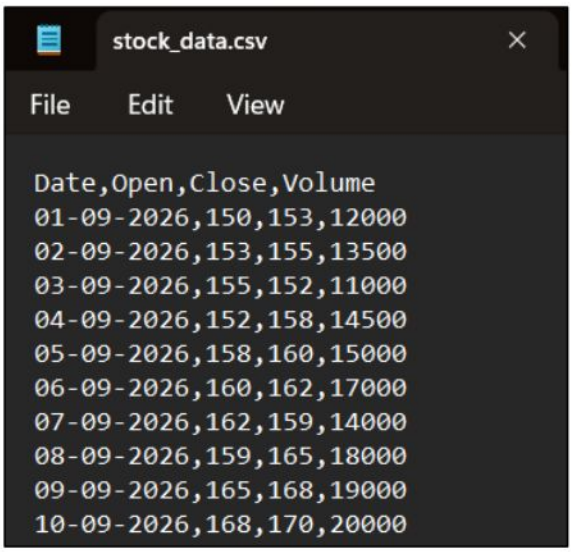
Description:

Read stock prices from CSV files and visualize opening price, closing price, trading volume, and moving averages.

Concepts Covered

- Data Visualization
- Line Graphs

Input:




```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("stock_data.csv")

df["Date"] = pd.to_datetime(
    df["Date"],
    format="%d-%m-%Y"
)

df["Moving_Average"] = df["Close"].rolling(window=3).mean()

print("===== STOCK MARKET DATA =====")
print(df)

print("\n===== STOCK STATISTICS =====")
print("Average Opening Price :",
      round(df["Open"].mean(), 2))
print("Average Closing Price :",
      round(df["Close"].mean(), 2))
print("Highest Closing Price :",
      df["Close"].max())
print("Lowest Closing Price  :",
      df["Close"].min())
print("Total Trading Volume  :",
      df["Volume"].sum())
```

```
# 1. OPENING AND CLOSING PRICE
plt.figure(figsize=(10, 5))
plt.plot(
    df["Date"],
    df["Open"],
    marker="o",
    label="Opening Price"
)
plt.plot(
    df["Date"],
    df["Close"],
    marker="o",
    label="Closing Price"
)
plt.title("Stock Opening and Closing Prices")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 2. TRADING VOLUME
plt.figure(figsize=(10, 5))
plt.plot(
    df["Date"],
    df["Volume"],
    marker="o"
)
plt.title("Daily Trading Volume")
plt.xlabel("Date")
plt.ylabel("Trading Volume")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
# 3. CLOSING PRICE AND MOVING AVERAGE
plt.figure(figsize=(10, 5))
plt.plot(
    df["Date"],
    df["Close"],
    marker="o",
    label="Closing Price"
)
plt.plot(
    df["Date"],
    df["Moving_Average"],
    marker="o",
    label="3-Day Moving Average"
)
plt.title("Stock Closing Price and Moving Average")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

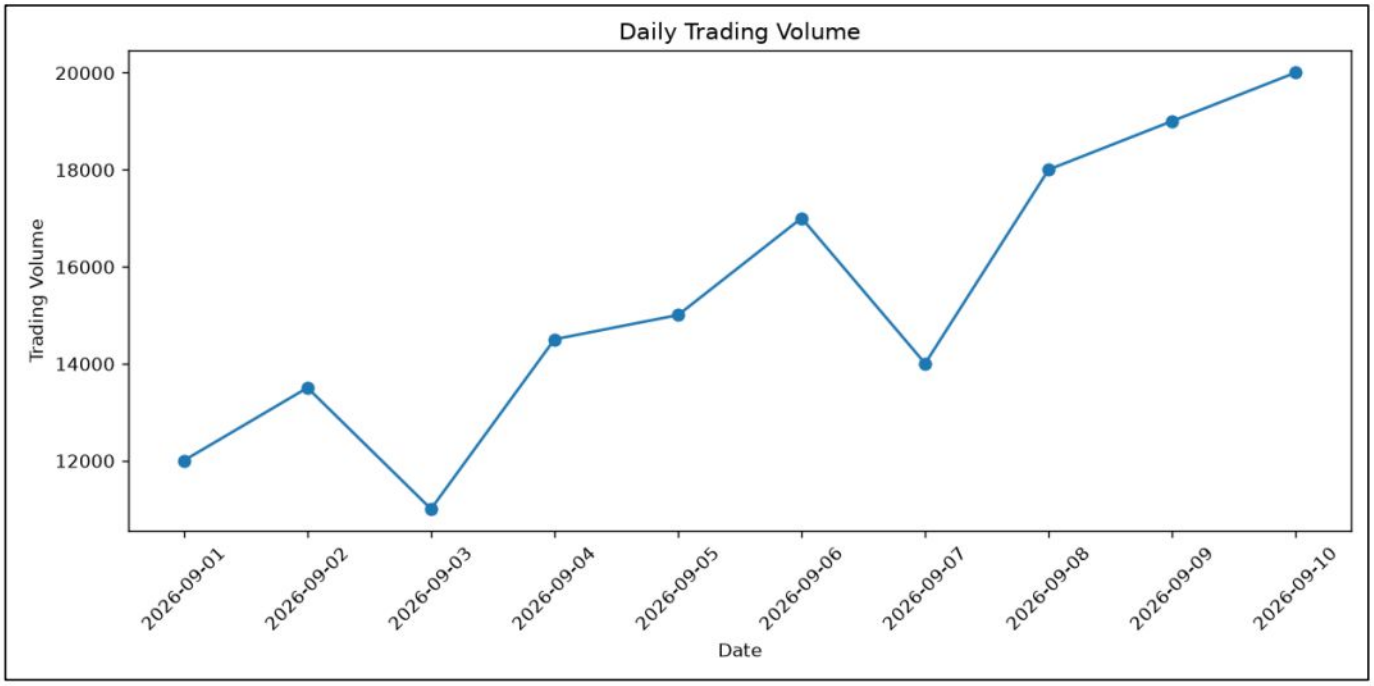
Output:

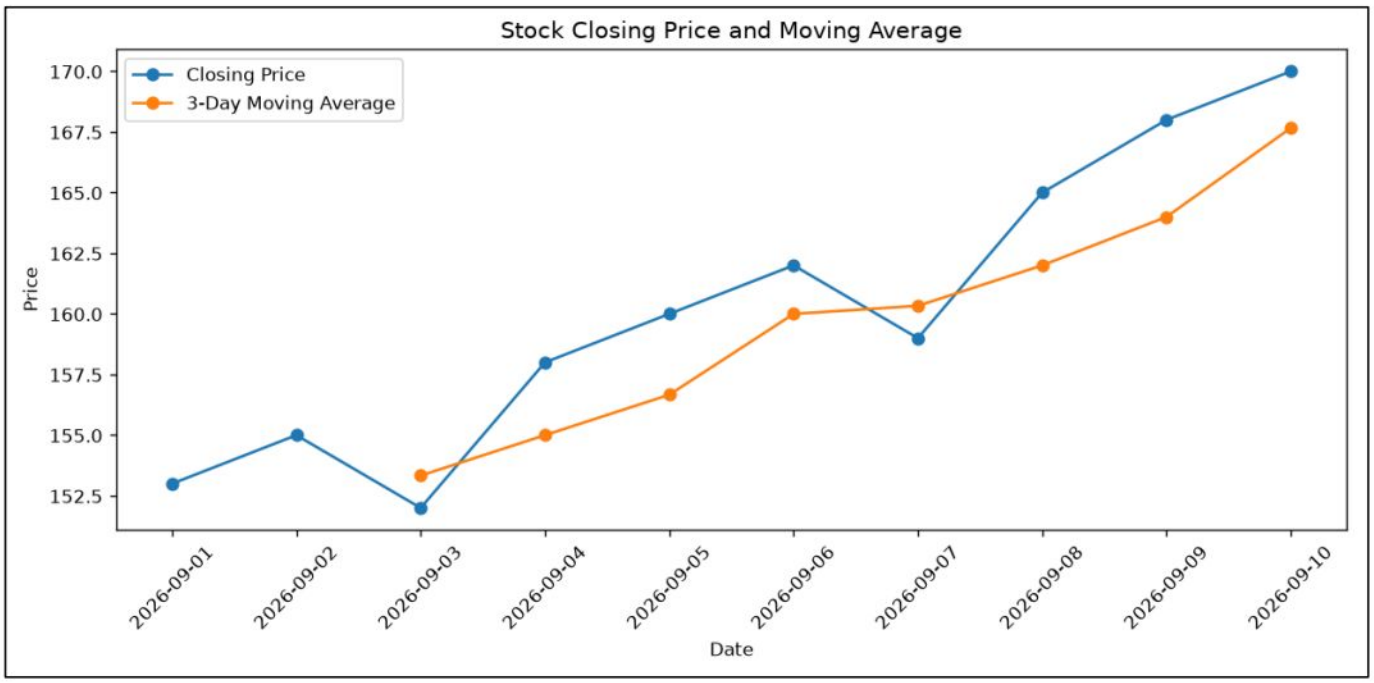
```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== STOCK MARKET DATA =====
        Date  Open  Close  Volume  Moving_Average
0 2026-09-01   150    153   12000             NaN
1 2026-09-02   153    155   13500             NaN
2 2026-09-03   155    152   11000      153.333333
3 2026-09-04   152    158   14500      155.000000
4 2026-09-05   158    160   15000      156.666667
5 2026-09-06   160    162   17000      160.000000
6 2026-09-07   162    159   14000      160.333333
7 2026-09-08   159    165   18000      162.000000
8 2026-09-09   165    168   19000      164.000000
9 2026-09-10   168    170   20000      167.666667

===== STOCK STATISTICS =====
Average Opening Price : 158.2
Average Closing Price : 160.2
Highest Closing Price : 170
Lowest Closing Price  : 152
Total Trading Volume  : 154000
```




Daily Trading Volume
Trading Volume
20000
18000
16000
14000
12000
2026-09-01
2026-09-02
2026-09-03
2026-09-04
2026-09-05
2026-09-06
2026-09-07
2026-09-08
2026-09-09
2026-09-10
Date


Stock Closing Price and Moving Average
Closing Price
3-Day Moving Average
Price
170.0
167.5
165.0
162.5
160.0
157.5
155.0
152.5
2026-09-01
2026-09-02
2026-09-03
2026-09-04
2026-09-05
2026-09-06
2026-09-07
2026-09-08
2026-09-09
2026-09-10
Date

Unit 6: Regular Expressions

Practical 1: Email Address Validator

Description:
Validate email addresses entered by users using regular expressions.

Concepts Covered
- Regular Expressions
- Pattern Matching

Input:

```
print("Name             :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import re

pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"

email = input("Enter your email address: ")

if re.fullmatch(pattern, email):
    print("Valid Email Address")
else:
    print("Invalid Email Address")
```

Output:

```
Name             :- JETHAVA JAY
Enrollment No. :- 250283152004

Enter your email address: jethavajay2007@gmail.com
Valid Email Address
```

Practical 2: Mobile Number Verification System

Description:

Validate Indian mobile numbers with country codes using regular expressions.

Concepts Covered

- Regex Quantifiers
- Character Classes

Input:

```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import re

pattern = r"^(?:\+91|91)?[- ]?[6-9][0-9]{9}$"

mobile = input("Enter Indian Mobile Number: ")

mobile = mobile.strip()

if re.fullmatch(pattern, mobile):
    print("Valid Indian Mobile Number")
else:
    print("Invalid Indian Mobile Number")
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

Enter Indian Mobile Number: 8160781243
Valid Indian Mobile Number
```

Practical 3: Password Policy Checker

Description:

Verify whether passwords satisfy organizational security policies using regular expressions.

Concepts Covered

- Regex
- Special Characters

Input:

```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import re

password = input("Enter your password: ")

pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[@#$%^&*!]).{8,}$"

if re.fullmatch(pattern, password):
    print("Valid Password")
    print("Password satisfies all security policies.")
else:
    print("Invalid Password")
    print("Password does not satisfy the security policies.")
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

Enter your password: Jay@1234
Valid Password
Password satisfies all security policies.
```

Practical 4: HTML Information Extractor

Description:

Extract hyperlinks, email addresses, and headings from HTML pages using regular expressions.

Concepts Covered

- Regex
- HTML Parsing

Input:

```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import re

html = """
<html>
<head>
    <title>My Web Page</title>
</head>
<body>
    <h1>Welcome to My Website</h1>
    <h2>About Us</h2>
    <h3>Contact Information</h3>
    <p>
        Visit our website:
        <a href="https://www.example.com">Example</a>
    </p>
    <p>
        Follow us at:
        <a href="https://www.python.org">Python</a>
    </p>
    <p>Email: contact@example.com</p>
    <p>Support: support@gmail.com</p>
</body>
</html>
"""

links = re.findall(r'href=[\"\'](.*?)[\"\']', html)

emails = re.findall(
    r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}',
    html
)

headings = re.findall(
    r'<h[1-6]>(.*?)</h[1-6]>',
    html
)
```

```
print("===== HTML INFORMATION EXTRACTOR =====")

print("\n--- HYPERLINKS ---")
for link in links:
    print(link)

print("\n--- EMAIL ADDRESSES ---")
for email in emails:
    print(email)

print("\n--- HEADINGS ---")
for heading in headings:
    print(heading)
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== HTML INFORMATION EXTRACTOR =====

--- HYPERLINKS ---
https://www.example.com
https://www.python.org

--- EMAIL ADDRESSES ---
contact@example.com
support@gmail.com

--- HEADINGS ---
Welcome to My Website
About Us
Contact Information
```

Practical 5: Resume Information Extractor

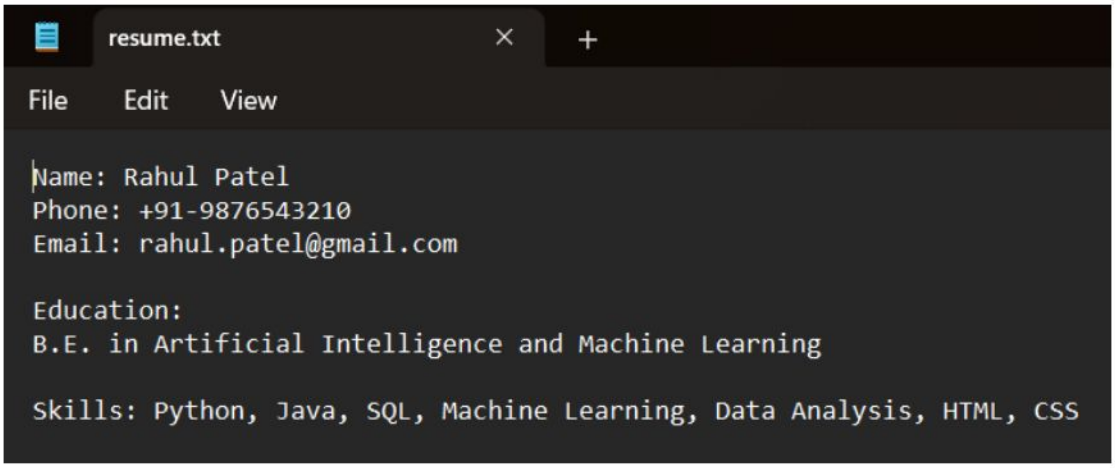
Description:

Extract candidate name, phone number, email address, and skills from resume text files using regular expressions.

Concepts Covered

- Regex
- File Handling

Input:




```python
print("Name             :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import re

FILE_NAME = "resume.txt"

try:
    with open(FILE_NAME, "r") as file:
        resume_text = file.read()

    name_match = re.search(
        r"(?i)Name\s*:\s*([A-Za-z ]+)",
        resume_text
    )
    phone_match = re.search(
        r"(?:\+91[- ]?)?[6-9][0-9]{9}",
        resume_text
    )
    email_match = re.search(
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
        resume_text
    )
    skills_match = re.search(
        r"(?i)Skills\s*:\s*(.+)",
        resume_text
    )
```

```
    print("===== RESUME INFORMATION EXTRACTOR =====")

    if name_match:
        print("Candidate Name :", name_match.group(1).strip())
    else:
        print("Candidate Name : Not Found")

    if phone_match:
        print("Phone Number   :", phone_match.group())
    else:
        print("Phone Number   : Not Found")

    if email_match:
        print("Email Address  :", email_match.group())
    else:
        print("Email Address  : Not Found")

    if skills_match:
        skills = skills_match.group(1).split(",")

        print("Skills         :")
        for skill in skills:
            print(" -", skill.strip())
    else:
        print("Skills         : Not Found")

except FileNotFoundError:
    print("Error: resume.txt file not found.")

except Exception as e:
    print("Error:", e)
```

Output:

```
Name            :- JETHAVA JAY
Enrollment No. :- 250283152004

===== RESUME INFORMATION EXTRACTOR =====
Candidate Name : Rahul Patel
Phone Number    : +91-9876543210
Email Address   : rahul.patel@gmail.com
Skills          :
 - Python
 - Java
 - SQL
 - Machine Learning
 - Data Analysis
 - HTML
 - CSS
```

Practical 6: Web Log Pattern Analyzer

Description:

Analyze web server log files and extract IP addresses, timestamps, request methods, and status codes using regular expressions.

Concepts Covered

- Regex
- File Processing

Input:

```
print("Name            :- JETHAVA JAY")
print("Enrollment No. :- 250283152004\n")

import re

FILE_NAME = "webserver.log"

try:
    with open(FILE_NAME, "r") as file:
        log_data = file.read()

    ip_addresses = re.findall(
        r'\b(?:\d{1,3}\.){3}\d{1,3}\b',
        log_data
    )
    timestamps = re.findall(
        r'\[([^\]]+)\]',
        log_data
    )
    methods = re.findall(
        r'"(GET|POST|PUT|DELETE|PATCH|HEAD|OPTIONS) ',
        log_data
    )
    status_codes = re.findall(
        r'"\s(\d{3})\s',
        log_data
    )

    print("===== WEB LOG PATTERN ANALYZER =====")
    print("\n--- IP ADDRESSES ---")
    for ip in ip_addresses:
        print(ip)
    print("\n--- TIMESTAMPS ---")
    for timestamp in timestamps:
        print(timestamp)
    print("\n--- REQUEST METHODS ---")
    for method in methods:
        print(method)
    print("\n--- STATUS CODES ---")
    for status in status_codes:
        print(status)

except FileNotFoundError:
    print("Error: webserver.log file not found.")

except Exception as e:
    print("Error:", e)
```

webserver.log

File Edit View

```
192.168.1.10 - - [02/Oct/2026:10:15:32 +0530] "GET /index.html HTTP/1.1" 200 1024
192.168.1.20 - - [02/Oct/2026:10:16:45 +0530] "POST /login HTTP/1.1" 302 512
10.0.0.5 - - [02/Oct/2026:10:17:20 +0530] "GET /products HTTP/1.1" 404 256
172.16.0.8 - - [02/Oct/2026:10:18:10 +0530] "DELETE /user/10 HTTP/1.1" 500 128
192.168.1.15 - - [02/Oct/2026:10:19:05 +0530] "PUT /profile HTTP/1.1" 200 768
```

Output:

```
Name             :- JETHAVA JAY
Enrollment No. :- 250283152004

===== WEB LOG PATTERN ANALYZER =====

--- IP ADDRESSES ---
192.168.1.10
192.168.1.20
10.0.0.5
172.16.0.8
192.168.1.15

--- TIMESTAMPS ---
02/Oct/2026:10:15:32 +0530
02/Oct/2026:10:16:45 +0530
02/Oct/2026:10:17:20 +0530
02/Oct/2026:10:18:10 +0530
02/Oct/2026:10:19:05 +0530

--- REQUEST METHODS ---
GET
POST
GET
DELETE
PUT

--- STATUS CODES ---
200
302
404
500
200
```

Unit 7: Python Database Connectivity

Practical 1: Student Database Management System

Description:

Develop a Python application that performs Create, Read, Update, and Delete (CRUD) operations on a MySQL student database.

Concepts Covered

- MySQL Connector
- CRUD Operations

Input:

```
import mysql.connector

# Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root"
)

cursor = conn.cursor()

# Create database
cursor.execute("CREATE DATABASE IF NOT EXISTS college")

# Select database
conn.database = "college"

# Create student table
cursor.execute("""
CREATE TABLE IF NOT EXISTS student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    course VARCHAR(50),
    email VARCHAR(100)
)
""")

# CREATE - Add student
def add_student():
    id = int(input("Enter Student ID: "))
    name = input("Enter Name: ")
    age = int(input("Enter Age: "))
    course = input("Enter Course: ")
    email = input("Enter Email: ")
```

```
    query = """
    INSERT INTO student (id, name, age, course, email)
    VALUES (%s, %s, %s, %s, %s)
    """

    values = (id, name, age, course, email)

    try:
        cursor.execute(query, values)
        conn.commit()
        print("Student added successfully!")
    except mysql.connector.Error as e:
        print("Error:", e)

# READ - Display students
def display_students():
    cursor.execute("SELECT * FROM student")
    records = cursor.fetchall()

    if not records:
        print("No student records found.")
    else:
        print("\nID\tName\tAge\tCourse\tEmail")
        print("-" * 60)

        for row in records:
            print(row[0], "\t", row[1], "\t", row[2],"\t", row[3], "\t", row[4])

# UPDATE - Update student
def update_student():
    id = int(input("Enter Student ID to update: "))

    name = input("Enter new Name: ")
    age = int(input("Enter new Age: "))
    course = input("Enter new Course: ")
    email = input("Enter new Email: ")

    query = """
    UPDATE student
    SET name=%s, age=%s, course=%s, email=%s
    WHERE id=%s
    """

    values = (name, age, course, email, id)

    cursor.execute(query, values)
    conn.commit()

    if cursor.rowcount > 0:
        print("Student updated successfully!")
    else:
        print("Student ID not found.")

# DELETE - Delete student
```

```
def delete_student():
    id = int(input("Enter Student ID to delete: "))

    query = "DELETE FROM student WHERE id=%s"

    cursor.execute(query, (id,))
    conn.commit()

    if cursor.rowcount > 0:
        print("Student deleted successfully!")
    else:
        print("Student ID not found.")

# Main menu
while True:
    print("\n===== STUDENT DATABASE MANAGEMENT SYSTEM
=====")
    print("1. Add Student")
    print("2. Display Students")
    print("3. Update Student")
    print("4. Delete Student")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()

    elif choice == "2":
        display_students()

    elif choice == "3":
        update_student()

    elif choice == "4":
        delete_student()

    elif choice == "5":
        print("Program terminated.")
        break

    else:
        print("Invalid choice!")

cursor.close()
conn.close()
```

- **Output:**

```
===== STUDENT DATABASE MANAGEMENT SYSTEM =====
1. Add Student
2. Display Students
3. Update Student
4. Delete Student
```

```
5. Exit
Enter your choice: 1
Enter Student ID: 007
Enter Name: jay
Enter Age: 20
Enter Course: AIML
Enter Email: jay@gmail.com
Student added successfully!

===== STUDENT DATABASE MANAGEMENT SYSTEM =====
1. Add Student
2. Display Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter Student ID: 105
Enter Name: om
Enter Age: 21
Enter Course: AIML
Enter Email: om@gmail.com
Student added successfully!

===== STUDENT DATABASE MANAGEMENT SYSTEM =====
1. Add Student
2. Display Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 2
```

ID	Name	Age	Course	Email
007	jay	20	AIML	jay@gmail.com
105	om	21	AIML	om@gmail.com

Practical 2: Library Database System

Description:

Create a library database and manage book records through Python.

Concepts Covered

- Database Connectivity
- SQL Queries

Input:

```
import mysql.connector

# Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root"
)

cursor = conn.cursor()

# Create database
cursor.execute("CREATE DATABASE IF NOT EXISTS library")
cursor.execute("USE library")

# Create books table
cursor.execute("""
CREATE TABLE IF NOT EXISTS books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100),
    author VARCHAR(100),
    price FLOAT
)
""")

# Insert book records
books = [
    (1, "Python Programming", "John Smith", 450),
    (2, "Data Science", "David Brown", 550),
    (3, "Machine Learning", "Andrew Ng", 650)
]

cursor.executemany("""
INSERT IGNORE INTO books (book_id, title, author, price)
VALUES (%s, %s, %s, %s)
""", books)

conn.commit()

# Display all books
print("\nLibrary Books:")
```

```
cursor.execute("SELECT * FROM books")

for book in cursor.fetchall():
    print(book)

# Update book price
cursor.execute("""
UPDATE books
SET price = 500
WHERE book_id = 1
""")

conn.commit()

print("\nAfter Updating Book:")
cursor.execute("SELECT * FROM books")

for book in cursor.fetchall():
    print(book)

# Delete a book
cursor.execute("""
DELETE FROM books
WHERE book_id = 3
""")

conn.commit()

print("\nAfter Deleting Book:")
cursor.execute("SELECT * FROM books")

for book in cursor.fetchall():
    print(book)

# Close connection
cursor.close()
conn.close()

print("\nDatabase connection closed.")
```

Output:

```
Library Books:
(1, 'Python Programming', 'John Smith', 500.0)
(2, 'Data Science', 'David Brown', 550.0)
(3, 'Machine Learning', 'Andrew Ng', 650.0)

After Updating Book:
(1, 'Python Programming', 'John Smith', 500.0)
(2, 'Data Science', 'David Brown', 550.0)
(3, 'Machine Learning', 'Andrew Ng', 650.0)

After Deleting Book:
(1, 'Python Programming', 'John Smith', 500.0)
(2, 'Data Science', 'David Brown', 550.0)

Database connection closed.
```

	book_id	title	author	price
Edit Copy Delete	1	Python Programming	John Smith	500
Edit Copy Delete	2	Data Science	David Brown	550

Practical 3: Employee Payroll Database

Description:

Store employee details in MySQL and generate payroll reports using Python.

Concepts Covered

- Insert
- Update
- Select

Input:

```
import mysql.connector

# Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root"
)

cursor = conn.cursor()

# Create database
cursor.execute("CREATE DATABASE IF NOT EXISTS payroll")
cursor.execute("USE payroll")

# Create employee table
cursor.execute("""
CREATE TABLE IF NOT EXISTS employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    salary FLOAT
)
""")

# Insert employee records
employees = [
    (1, "Rahul", "IT", 50000),
    (2, "Priya", "HR", 45000),
    (3, "Amit", "Finance", 55000)
]

cursor.executemany("""
INSERT IGNORE INTO employees
(emp_id, name, department, salary)
VALUES (%s, %s, %s, %s)
""", employees)

conn.commit()
```

```
# Display employee records
print("\nEmployee Payroll Report:")
cursor.execute("SELECT * FROM employees")

for employee in cursor.fetchall():
    print(employee)

# Update salary
cursor.execute("""
UPDATE employees
SET salary = 60000
WHERE emp_id = 1
""")

conn.commit()

# Display updated records
print("\nAfter Salary Update:")
cursor.execute("SELECT * FROM employees")

for employee in cursor.fetchall():
    print(employee)

# Generate payroll report
print("\nFinal Payroll Report:")
cursor.execute("""
SELECT emp_id, name, department, salary
FROM employees
""")

for employee in cursor.fetchall():
    print(
        "ID:", employee[0],
        "| Name:", employee[1],
        "| Department:", employee[2],
        "| Salary:", employee[3]
    )

# Close connection
cursor.close()
conn.close()

print("\nDatabase connection closed.")
```

Output:

```
Employee Payroll Report:
(1, 'Rahul', 'IT', 60000.0)
(2, 'Priya', 'HR', 45000.0)
(3, 'Amit', 'Finance', 55000.0)

After Salary Update:
(1, 'Rahul', 'IT', 60000.0)
(2, 'Priya', 'HR', 45000.0)
(3, 'Amit', 'Finance', 55000.0)

Final Payroll Report:
ID: 1 | Name: Rahul | Department: IT | Salary: 60000.0
ID: 2 | Name: Priya | Department: HR | Salary: 45000.0
ID: 3 | Name: Amit | Department: Finance | Salary: 55000.0

Database connection closed.
```

←T→				emp_id	name	department	salary
☐	Edit	Copy	Delete	1	Rahul	IT	60000
☐	Edit	Copy	Delete	2	Priya	HR	45000
☐	Edit	Copy	Delete	3	Amit	Finance	55000

Practical 4: Hospital Patient Record Management

Description:

Maintain patient records in MySQL with options for registration, searching, updating, and deleting records.

Concepts Covered

- Database Programming

Input:

```
import mysql.connector

# Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root"
)

cursor = conn.cursor()

# Create database
cursor.execute("CREATE DATABASE IF NOT EXISTS hospital")
cursor.execute("USE hospital")

# Create patient table
cursor.execute("""
CREATE TABLE IF NOT EXISTS patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT,
    gender VARCHAR(10),
    disease VARCHAR(100)
)
""")

# Function to register patient
def register_patient():
    patient_id = int(input("Enter Patient ID: "))
    name = input("Enter Patient Name: ")
    age = int(input("Enter Age: "))
    gender = input("Enter Gender: ")
    disease = input("Enter Disease: ")

    query = """
    INSERT INTO patients
    (patient_id, name, age, gender, disease)
    VALUES (%s, %s, %s, %s, %s)
    """

    cursor.execute(query, (patient_id, name, age, gender, disease))
```

```
    conn.commit()

    print("Patient registered successfully.")

# Function to search patient
def search_patient():
    patient_id = int(input("Enter Patient ID to search: "))

    cursor.execute(
        "SELECT * FROM patients WHERE patient_id = %s",
        (patient_id,)
    )

    patient = cursor.fetchone()

    if patient:
        print("\nPatient Record:")
        print("ID:", patient[0])
        print("Name:", patient[1])
        print("Age:", patient[2])
        print("Gender:", patient[3])
        print("Disease:", patient[4])
    else:
        print("Patient not found.")

# Function to update patient
def update_patient():
    patient_id = int(input("Enter Patient ID to update: "))
    disease = input("Enter new disease: ")

    cursor.execute(
        "UPDATE patients SET disease = %s WHERE patient_id = %s",
        (disease, patient_id)
    )

    conn.commit()

    if cursor.rowcount > 0:
        print("Patient record updated successfully.")
    else:
        print("Patient not found.")

# Function to delete patient
def delete_patient():
    patient_id = int(input("Enter Patient ID to delete: "))

    cursor.execute(
        "DELETE FROM patients WHERE patient_id = %s",
        (patient_id,)
    )

    conn.commit()
```

```
    if cursor.rowcount > 0:
        print("Patient record deleted successfully.")
    else:
        print("Patient not found.")

# Main menu
while True:
    print("\n===== Hospital Patient Record Management =====")
    print("1. Register Patient")
    print("2. Search Patient")
    print("3. Update Patient")
    print("4. Delete Patient")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        register_patient()

    elif choice == "2":
        search_patient()

    elif choice == "3":
        update_patient()

    elif choice == "4":
        delete_patient()

    elif choice == "5":
        print("Exiting program...")
        break

    else:
        print("Invalid choice.")

# Close connection
cursor.close()
conn.close()
```

Output:

```
===== Hospital Patient Record Management =====
1. Register Patient
2. Search Patient
3. Update Patient
4. Delete Patient
5. Exit
Enter your choice: 1
Enter Patient ID: 007
Enter Patient Name: jay
Enter Age: 30
Enter Gender: Male
Enter Disease: Fever
Patient registered successfully.

===== Hospital Patient Record Management =====
1. Register Patient
2. Search Patient
3. Update Patient
4. Delete Patient
5. Exit
Enter your choice: 1
Enter Patient ID: 105
Enter Patient Name: om
Enter Age: 20
Enter Gender: Male
Enter Disease: Fever
Patient registered successfully.

===== Hospital Patient Record Management =====
1. Register Patient
2. Search Patient
3. Update Patient
4. Delete Patient
5. Exit
Enter your choice: 3
Enter Patient ID to update: 007
Enter new disease: Cancers
Patient record updated successfully.
```

```
===== Hospital Patient Record Management =====
1. Register Patient
2. Search Patient
3. Update Patient
4. Delete Patient
5. Exit
Enter your choice: 2
Enter Patient ID to search: 105

Patient Record:
ID: 105
Name: om
Age: 20
Gender: Male
Disease: Fever

===== Hospital Patient Record Management =====
1. Register Patient
2. Search Patient
3. Update Patient
4. Delete Patient
5. Exit
Enter your choice: 4
Enter Patient ID to delete: 105
Patient record deleted successfully.
```

```
===== Hospital Patient Record Management =====
1. Register Patient
2. Search Patient
3. Update Patient
4. Delete Patient
5. Exit
Enter your choice: 5
Exiting program.
```

patient_id	name	age	gender	disease
007	jay	30	Male	Cancers

Practical 5: Online Food Ordering Database

Description:

Develop a Python application that manages customers, menu items, and orders using a MySQL database.

Concepts Covered

- Database Connectivity
- Transactions

Input:

```
import mysql.connector

# Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password"
)

cursor = conn.cursor()

# Create database
cursor.execute("CREATE DATABASE IF NOT EXISTS food_ordering")
cursor.execute("USE food_ordering")

# Create Customers table
cursor.execute("""
CREATE TABLE IF NOT EXISTS customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100),
    phone VARCHAR(15)
)
""")

# Create Menu table
cursor.execute("""
CREATE TABLE IF NOT EXISTS menu (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100),
    price FLOAT
)
""")

# Create Orders table
cursor.execute("""
CREATE TABLE IF NOT EXISTS orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    item_id INT,
    quantity INT,
    total_amount FLOAT,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
```

```
    FOREIGN KEY (item_id) REFERENCES menu(item_id)
)
""")

# Insert customers
customers = [
    (1, "Rahul", "9876543210"),
    (2, "Priya", "9876501234"),
    (3, "Amit", "9988776655")
]

cursor.executemany("""
INSERT IGNORE INTO customers
(customer_id, name, phone)
VALUES (%s, %s, %s)
""", customers)

# Insert menu items
menu_items = [
    (1, "Pizza", 250),
    (2, "Burger", 150),
    (3, "Pasta", 200),
    (4, "Sandwich", 120)
]

cursor.executemany("""
INSERT IGNORE INTO menu
(item_id, item_name, price)
VALUES (%s, %s, %s)
""", menu_items)

conn.commit()

# Display menu
print("\n===== MENU =====")

cursor.execute("SELECT * FROM menu")

for item in cursor.fetchall():
    print(item)

# Place an order using transaction
try:
    order_id = 101
    customer_id = 1
    item_id = 1
    quantity = 2

    # Get item price
    cursor.execute(
        "SELECT price FROM menu WHERE item_id = %s",
        (item_id,)
```

```
    )

    result = cursor.fetchone()

    if result is None:
        print("Food item not found.")
    else:
        price = result[0]
        total_amount = price * quantity

        # Insert order
        cursor.execute("""
        INSERT INTO orders
        (order_id, customer_id, item_id, quantity, total_amount)
        VALUES (%s, %s, %s, %s, %s)
        """, (
            order_id,
            customer_id,
            item_id,
            quantity,
            total_amount
        ))

        # Commit transaction
        conn.commit()

        print("\nOrder placed successfully.")
        print("Order ID:", order_id)
        print("Total Amount:", total_amount)

except Exception as e:
    # Rollback transaction if error occurs
    conn.rollback()
    print("Transaction failed.")
    print("Error:", e)

# Display orders
print("\n===== ORDERS =====")

cursor.execute("""
SELECT
    orders.order_id,
    customers.name,
    menu.item_name,
    orders.quantity,
    orders.total_amount
FROM orders
JOIN customers
ON orders.customer_id = customers.customer_id
JOIN menu
ON orders.item_id = menu.item_id
""")
```

```
    for order in cursor.fetchall():
        print(order)

    # Close connection
    cursor.close()
    conn.close()

    print("\nDatabase connection closed.")
```

Output:

```
===== MENU =====
(1, 'Pizza', 250.0)
(2, 'Burger', 150.0)
(3, 'Pasta', 200.0)
(4, 'Sandwich', 120.0)
Transaction failed.
Error: 1062 (23000): Duplicate entry '101' for key 'PRIMARY'

===== ORDERS =====
(101, 'Rahul', 'Pizza', 2, 500.0)

Database connection closed.
```

Practical 6: College Examination Management System

Description:

Create a database-driven application for storing examination results, retrieving student marks, generating grade reports, and updating examination records.

Concepts Covered

- Python-MySQL Connectivity
- CRUD Operations
- Database Tables

Input:

```
import mysql.connector

# Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root"
)

cursor = conn.cursor()

# Create database
cursor.execute("CREATE DATABASE IF NOT EXISTS college_exam")
cursor.execute("USE college_exam")

# Create examination table
cursor.execute("""
CREATE TABLE IF NOT EXISTS results (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    subject VARCHAR(50),
    marks INT,
    grade VARCHAR(5)
)
""")

# Function to calculate grade
def calculate_grade(marks):
    if marks >= 90:
        return "A+"
    elif marks >= 80:
        return "A"
    elif marks >= 70:
        return "B"
    elif marks >= 60:
        return "C"
    elif marks >= 50:
        return "D"
    else:
```

```
        return "F"

# CREATE - Add examination record
def add_result():
    student_id = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")
    subject = input("Enter Subject: ")
    marks = int(input("Enter Marks: "))

    grade = calculate_grade(marks)

    query = """
    INSERT INTO results
    (student_id, name, subject, marks, grade)
    VALUES (%s, %s, %s, %s, %s)
    """

    cursor.execute(
        query,
        (student_id, name, subject, marks, grade)
    )

    conn.commit()

    print("Examination record added successfully.")

# READ - Display all results
def display_results():
    cursor.execute("SELECT * FROM results")

    records = cursor.fetchall()

    print("\n===== EXAMINATION RESULTS =====")

    if records:
        for record in records:
            print(
                "ID:", record[0],
                "| Name:", record[1],
                "| Subject:", record[2],
                "| Marks:", record[3],
                "| Grade:", record[4]
            )
    else:
        print("No records found.")

# READ - Search student result
def search_result():
    student_id = int(input("Enter Student ID: "))

    cursor.execute(
        "SELECT * FROM results WHERE student_id = %s",
```

```
        (student_id,)
    )

    record = cursor.fetchone()

    if record:
        print("\n===== STUDENT GRADE REPORT =====")
        print("Student ID:", record[0])
        print("Name:", record[1])
        print("Subject:", record[2])
        print("Marks:", record[3])
        print("Grade:", record[4])
    else:
        print("Student record not found.")

# UPDATE - Update examination marks
def update_result():
    student_id = int(input("Enter Student ID: "))
    marks = int(input("Enter New Marks: "))

    grade = calculate_grade(marks)

    cursor.execute("""
        UPDATE results
        SET marks = %s, grade = %s
        WHERE student_id = %s
    """, (marks, grade, student_id))

    conn.commit()

    if cursor.rowcount > 0:
        print("Examination record updated successfully.")
    else:
        print("Student record not found.")

# DELETE - Delete examination record
def delete_result():
    student_id = int(input("Enter Student ID: "))

    cursor.execute(
        "DELETE FROM results WHERE student_id = %s",
        (student_id,)
    )

    conn.commit()

    if cursor.rowcount > 0:
        print("Examination record deleted successfully.")
    else:
        print("Student record not found.")

# Main menu
```

```
while True:

    print("\n===== COLLEGE EXAMINATION MANAGEMENT =====")
    print("1. Add Examination Result")
    print("2. Display All Results")
    print("3. Search Student Result")
    print("4. Update Examination Record")
    print("5. Delete Examination Record")
    print("6. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_result()

    elif choice == "2":
        display_results()

    elif choice == "3":
        search_result()

    elif choice == "4":
        update_result()

    elif choice == "5":
        delete_result()

    elif choice == "6":
        print("Exiting program...")
        break

    else:
        print("Invalid choice. Please try again.")

# Close database connection
cursor.close()
conn.close()

print("Database connection closed.")
```

Output:

```
===== COLLEGE EXAMINATION MANAGEMENT =====
1. Add Examination Result
2. Display All Results
3. Search Student Result
4. Update Examination Record
5. Delete Examination Record
6. Exit
Enter your choice: 1
Enter Student ID: 007
Enter Student Name: jay
Enter Subject: Machine Learning
Enter Marks: 97
Examination record added successfully.

===== COLLEGE EXAMINATION MANAGEMENT =====
1. Add Examination Result
2. Display All Results
3. Search Student Result
4. Update Examination Record
5. Delete Examination Record
6. Exit
Enter your choice: 1
Enter Student ID: 105
Enter Student Name: om
Enter Subject: Mathematics
Enter Marks: 95
Examination record added successfully.

===== COLLEGE EXAMINATION MANAGEMENT =====
1. Add Examination Result
2. Display All Results
3. Search Student Result
4. Update Examination Record
5. Delete Examination Record
6. Exit
Enter your choice: 2

===== EXAMINATION RESULTS =====
ID: 101 | Name:jay | Subject: Machine Learning | Marks: 97 | Grade: A+
ID: 102 | Name:om | Subject: Mathematics | Marks: 95 | Grade: A+
```

```
===== COLLEGE EXAMINATION MANAGEMENT =====
1. Add Examination Result
2. Display All Results
3. Search Student Result
4. Update Examination Record
5. Delete Examination Record
6. Exit
Enter your choice: 4
Enter Student ID: 105
Enter New Marks: 96
Examination record updated successfully.

===== COLLEGE EXAMINATION MANAGEMENT =====
1. Add Examination Result
2. Display All Results
3. Search Student Result
4. Update Examination Record
5. Delete Examination Record
6. Exit
Enter your choice: 5
Enter Student ID: 105
Examination record deleted successfully.
```

```
===== COLLEGE EXAMINATION MANAGEMENT =====
1. Add Examination Result
2. Display All Results
3. Search Student Result
4. Update Examination Record
5. Delete Examination Record
6. Exit
Enter your choice: 6
Exiting program...
Database connection closed.
```

student_id	name	subject	marks	grade
007	jay	Machine Learning	97	A+